

An Update-intensive LSM-based R-tree Index

Jaewoo Shin · Libin Zhou · Jianguo Wang · Walid G. Aref

Received: date / Accepted: date

Abstract Many applications require update-intensive workloads on spatial objects, e.g., social-network services and shared-riding services that track moving objects. By buffering insert and delete operations in memory, the Log Structured Merge Tree (LSM) has been used widely in various systems because of its ability to handle write-heavy workloads. While the focus on LSM has been on key-value stores and their optimizations, there is a need to study how to efficiently support LSM-based *secondary* indexes (e.g., location-based indexes) as modern, heterogeneous data necessitates the use of secondary indexes. In this paper, we investigate the augmentation of a main-memory-based memo structure into an LSM secondary index structure to handle update-intensive workloads efficiently. We conduct this study in the context of an R-tree-based secondary index. In particular, we introduce the LSM RUM-tree that demonstrates the use of an Update Memo in an LSM-based R-tree to enhance the performance of the R-tree's insert, delete, update, and search operations. The LSM RUM-tree introduces new strategies to control the size of the Update Memo to make sure it always fits in memory for high performance. The Update Memo is a light-weight in-memory structure that is suitable

for handling update-intensive workloads without introducing significant overhead. Experimental results using real spatial data demonstrate that the LSM RUM-tree achieves up to 6.6x speedup on update operations and up to 249292x speedup on query processing over existing LSM R-tree implementations.

Keywords LSM-based index · secondary index · query processing · R-tree · big data · spatial databases

1 Introduction

In recent years, massive amounts of location data have been generated continuously from mobile devices, social media, and shared-riding services. As devices or objects move in space, they update their locations and expect to have responsive services (e.g., getting weather emergencies and location-targeted advertisements). From a system's perspective, it is challenging to efficiently handle update-intensive location workloads, and answer queries with low latency.

A widely-used approach for write-intensive workloads is the Log-Structured Merge (LSM) tree [63]. The main idea of LSM is to buffer data ingestion in memory, and then periodically flush the buffers into disk. This can convert random I/Os to sequential I/Os for heavy write workloads. The LSM R-tree has been proposed to handle write-intensive spatial workloads [11]. Locations or coordinates are not a primary key, but rather a secondary key. The reason is that location data cannot uniquely identify an object in the same way an object identifier does. Moreover, a certain location can be shared by multiple objects at different times.

Determining the most recent state of an object (in this case, the object's current location) in write-intensive (update-intensive) workloads without much overhead

J. Shin
Purdue University, West Lafayette, IN
E-mail: shin152@purdue.edu

L. Zhou
Purdue University, West Lafayette, IN
E-mail: zhou822@purdue.edu

J. Wang
Purdue University, West Lafayette, IN
E-mail: csjgwang@purdue.edu

W. G. Aref
Purdue University, West Lafayette, IN
E-mail: aref@purdue.edu

is challenging. For example, consider a moving-objects spatial database application that maintains the current locations of objects moving in space. This application is update-intensive due to the continuous movements of the objects. An LSM-like structure for storing location data would be a perfect match. The LSM R-tree [11,58] has been introduced as a way to ingest object movements in space while at the same time being able to answer location-oriented queries, e.g., range queries. In the LSM R-tree, an object moving in space continuously updates the object’s location into an in-memory R-tree. There is only one “current” location for an object, and all the previous locations for the object become out-dated. When the allocated memory for the in-memory R-tree runs out, it is dumped to disk, and a new in-memory R-tree is started. When querying the R-tree, e.g., a range search query, all the in-memory and disk-based R-trees are probed, and hence the results may contain outdated locations of the object. Unless handled properly, this could lead to incorrect answers to the range query. Thus, additional measures are taken in the LSM R-tree to ensure correct results [11,58].

Alsubaiee et al. [11] address this issue via an *eager* strategy, where an additional data structure, namely, a deleted-key B^+ -tree, is associated with an in-memory R-tree to store the deleted objects’ keys. This indicates that an old version of the object is deleted, and a new one is inserted. The deleted-key B^+ -tree is also flushed to disk along with the corresponding R-tree. This scheme induces extra overhead due to the need to maintain and access the deleted-key B^+ -tree, and thus affects both the update and query performance.

Another solution to address the issue is to use a *validation* strategy [58], where the deleted-key B^+ -tree that is coupled with each R-tree is removed. Instead, a primary key index (i.e., a B^+ -tree holding the primary keys) with timestamps is used to lazily clean the obsolete objects during the merge operation of the LSM R-tree. The decoupled primary key index with timestamps helps avoid the extra maintenance cost, and shows improvement in update performance. However, this approach induces extra overhead in the search operation to validate the most recent state of an object. While enhancing update performance, this approach penalizes search in contrast to the eager strategy.

The RUM-tree [75,82] has been introduced where a disk-based R-tree handles frequently-updated location data using an in-memory structure, termed the Update Memo (UM), by maintaining the timestamp of the most-recent object-update in the UM. The UM has been utilized in the context of the R-tree [75,82,88], the Grid File [83], in indexing limited trajectories [60], and in indexing current and near-future locations of mov-

ing objects [22]. Because the UM keeps every update in memory, its size matters. In the RUM-tree [75,82] the UM size is controlled (upper-bounded) by probing the R-tree nodes in disk to lazily perform garbage cleaning. However, garbage cleaning strategies in the original RUM-tree and its variants are not directly applicable to an LSM-based R-tree index because in LSM, multiple R-trees exist on disk and store some of the obsolete (out-dated) locations of the moved objects. In this case, garbage cleaning would become expensive if applied without adaptation to the LSM environment. It is non-trivial to get rid of these out-dated objects in the R-tree, and thus it is more challenging to enforce an upper-bound on the size of the UM when deployed in an LSM-based index. Moreover, the original Update Memo is sensitive to the insert commands, i.e., it gets updated when an insert operation is performed on the accompanying index. This is not practical especially for insert-intensive workloads. Even if we maintain multiple deleted-key B^+ -trees [11] or a primary key index [58] in the memory layer, that would not be a sufficient solution for update-intensive workloads. The reason is that having multiple deleted-key B^+ -trees would require multi-path searches, and the maintenance cost for the orchestration with the LSM R-tree would still exist. Similarly, having a primary key index in memory does not scale. The reason is that as it may exceed the memory budget because the size of the primary key index is proportional to the total number of primary keys, which is not guaranteed to fit in memory.

We propose a new scheme that improves both the update and search performance. We adapt and investigate the use of the Update Memo in the context of an LSM secondary index, mainly the R-tree. We refer to the new UM-enabled index by the LSM RUM-tree.

The main contributions of this paper are as follows:

- We introduce the LSM RUM-Tree, an LSM-based R-tree that utilizes an in-memory Update Memo (UM) to support update-intensive spatial workloads without sacrificing search performance. The LSM RUM-Tree maintains multiple R-trees on disk but its UM component that resides only in memory is bounded in size (not proportional to the number of objects in the system). The UM is guaranteed not to exceed its allocated memory.
- The LSM RUM-Tree is presented in the context of geo-location data. However, the LSM-based Update Memo (UM) can be applied to other update-intensive secondary indexes, e.g., a B^+ -tree or an inverted index (Section 3).
- We introduce four cleaning strategies to bound the UM size in memory and the R-trees in disk. This

does not only reduce the consumed memory space, but also improves the overall performance (Section 4).

- We conduct experiments using real datasets to replicate update-intensive workloads. Results show that the LSM RUM-tree enhances the performance of the LSM-based R-tree on the delete, update, and search operations with up to 9.6x and 2400x speedups on update and search performance, respectively, over state-of-the-art LSM R-tree implementations.

This paper is an extended version of a short paper that introduces the LSM RUM-tree [74]. This paper extends over the short paper [74] in the following ways.

- We extend the analysis of LSM-aware UM cleaning strategies to show how well each of the UM cleaning strategies shrinks the UM size, and achieves performance improvements (Section 4).
- We introduce concurrency control for the Update Memo. This is essential to support concurrent activities over the LSM RUM-Tree, e.g., in a multi-threaded/multi-core setup. We introduce the atomic operation *Compare and If Less then Swap* (CILS) to facilitate concurrent activities on the UM in a multi-threaded environment (Section 5). The experiments show 4.5x speedup on concurrent update operations over the state-of-the-art implementations.
- Our comprehensive analysis of the LSM RUM-tree aims to demonstrate where performance improvements have been made over existing implementations. We include new additional LSM RUM-tree experiments over the ones in the original short paper, and discuss the experimental results (Section 6).

The rest of this paper proceeds as follows. Section 2 presents background material and discusses the related work. Section 3 introduces the LSM RUM-tree, all its supporting index operations, and shows how the Update Memo (UM) is adapted for use inside the LSM RUM-tree. Section 4 presents how the cleaning strategies bound the size of the UM, and improve the performance of the LSM RUM-tree. Section 5 discusses UM’s concurrent operations. Section 6 presents experimental results, and Section 7 concludes the paper.

2 Background and Related Work

We present an overview of the LSM-tree [63] along with two prior approaches to handle spatial workloads in the LSM R-tree. In order to improve performance of the LSM R-tree via the use of the Update Memo, one needs to understand how the LSM R-tree works for each operation and how the Update Memo can accelerate the update procedure for the R-tree.

2.1 The R-tree

The R-tree [35] is a B+-tree-like index that stores multi-dimensional/spatial objects. The R-tree stores spatial objects in leaf nodes according to their Minimum Bounding Rectangles (MBRs). Each R-tree leaf node stores an MBR that encapsulates all spatial objects within it, and each internal node stores an MBR encapsulating all its child nodes’ MBRs. R-tree nodes are organized and managed similarly as in a B+-tree, where each internal node stores keys (in terms of MBRs) and pointers to child nodes, and only leaf nodes store the actual objects. An R-tree node maintains similar balance requirement as a B+-tree node with splits during overflow and merge during underflow. The only exception is that R-tree nodes are allowed to overlap. That is, R-tree nodes’ MBRs may overlap. Therefore, R-tree querying requires checking all R-tree nodes with MBRs overlapping with the query range, which can be more than one path from root to leaf, in contrast to a B+-tree. The R-Tree supports updates through top-down approaches, e.g., [16, 35, 46], e.g., bottom-up approaches [51, 52], or in memory buffering/memo approaches, e.g., [75, 76, 82, 89].

2.2 Log-Structured Merge Tree

The LSM-tree [63] has been introduced to optimize write-intensive workloads by buffering data ingestion in the memory layer. Once the size of the data in the memory layer exceeds a certain threshold, the LSM-tree flushes the memory data to the disk layer using sequential I/O. As the flushed components accumulate in the disk layer, they get merged together based on a merge policy so that the LSM-tree maintains a limited number of disk components. Those disk components are managed in layers, and each deeper layer contains exponentially more components than the previous layer. Because of the LSM-tree’s buffering ability to handle write-intensive workloads efficiently, it has been widely used in a variety of key-value store systems, e.g., AsterixDB [1, 10, 11, 58], Google’s BigTable [20] and LevelDB [4], Cassandra [2], Amazon’s Dynamo [30], Apache’s HBase [3], and Accumulo [13], Alibaba’s X-Engine [39], LinkedIn’s Voldemort [55], Facebook’s RocksDB [6], and cLSM [34] and bLSM [73] at Yahoo.

There are also a large amount of research works on improving and optimizing LSM-tree’s performance. Sarkar and Athanassoulis [69] organize these works into the following categories: Optimization of the main memory part of the LSM-tree, e.g., as in [2, 4, 6, 7, 19, 32, 39], optimization of the disk components of the LSM-tree,

e.g., as in [21, 25, 27–29, 40, 42, 56, 59, 68, 81], disk components compaction optimization and design space navigation, e.g., as in [9, 10, 64, 70, 72, 84, 85], and improving parallelism and concurrency, e.g., as in [23, 32, 47, 53, 54, 77, 78, 80, 86, 87]. On the other hand, they identify another group of research works on navigating the design space and tuning LSM-tree configurations for optimal performance and tradeoffs, e.g., as in [14, 15, 18, 26, 28, 41, 43–45, 57, 59, 71]. For more information on (primary) LSM-tree research and application, we recommend checking out their recent tutorial [69].

The LSM-tree is commonly used as the primary storage in key-value stores. There has been research focus on taking advantage of its high write throughput to support efficient secondary indexes, e.g., for spatial objects, as in [8, 36, 48, 58, 61, 67, 74, 79], and for time series, e.g., as in [8, 49]. This paper targets moving spatial objects that are indexed and queried by their locations (and hence, resulting in a secondary key index) instead of indexing by object identifiers (primary keys).

For clarity, we use the following terms throughout the paper as stated in the previous studies. *Primary index* is an LSM-based B^+ -tree, where the key is the primary key (e.g., O_{id}) and the value is the data describing an object. *Primary key index* and *Deleted-key B^+ -tree* are also LSM-based B^+ -trees, but they hold the primary key only, not the entire object [11, 58]. These indexes can be seen as using a B^+ -tree as the memory component of an LSM tree (termed LSM B-tree). They store a B^+ -tree in memory, and once full, they flush the B^+ -trees and merge them similar to an LSM-tree’s disk component. The LSM secondary index (e.g., the LSM R-tree [11]) stores secondary keys (e.g., location) and values that are the primary key of the objects (the object IDs). Notice that there can be multiple objects having the same secondary keys (e.g., different objects could be placed in the same location). In this paper, our focus is on LSM secondary index updates instead of just writes. Our work targets moving objects, whose locations are constantly **changing**, indicating updates of existing objects in the LSM-tree. Moreover, our work is a specific optimization on supporting high update performance on the LSM R-tree (a secondary index explained in the next section). Similar to the original RUM-tree works [75, 82] that can be applied on any R-tree algorithms, our optimization can be directly applied on any LSM tree-based secondary indexes, e.g. other implementations of LSM R-tree [36, 61, 67].

2.3 The LSM R-tree

Traditional disk-based spatial indexes, e.g., the R-tree [35], adopt an “in-place update” policy. This re-

sults in many random I/Os. Given the high update rates of moving objects in space in the spatial data domain and the need to index these moving objects, it is not a surprise to apply LSM to the R-tree to deal with these update-intensive spatial workloads. The LSM R-tree [11] is a spatial index to handle location data efficiently, and benefits from the LSM mechanism. By applying a generic framework for secondary index *LSMification* [11], the LSM R-tree is an optimized secondary index for write-intensive spatial data workloads. The LSM R-tree adopts R-trees as the storage components (in memory and on disk) of an LSM-tree. The LSM R-tree processes and applies updates to the in-memory R-tree using regular R-tree operations. Once the main memory R-tree’s size reaches a memory-size threshold, the whole R-tree is flushed to disk as in the LSM-tree. A flush creates a new R-tree on disk, and each disk R-tree can be loaded back into DRAM for read operations as in the case of reading the LSM-tree disk components. When the number of disk R-tree components reach a merge threshold size, they are read and are merged and written into a new R-tree, and the new R-tree is written to a new file on disk sequentially. Thus, the LSM R-tree avoids in-place updates and improves write performance by having multiple disk R-trees. Read operations search both the in-memory R-tree and disk components for each query. However, due to storing multiple R-trees and obsolete objects on disk, without proper update maintenance, read queries in the LSM R-tree cannot determine the correct state of queried objects, or incurs high overheads [11, 58]. Thus, additional book-keeping in the LSM-tree is needed to resolve this issue.

Traditionally, updating a secondary key value, e.g., updating the location of an object, requires maintenance of both an LSM *primary* and *secondary* indexes to ensure consistency among both indexes. In the primary index side, the object ids (key) need to keep the new object location, while in the secondary index side, the new location (key) of an object should point to this object’s id. Maintaining the primary index is straightforward. It buffers updates into the component in the memory layer. However, the maintenance of the LSM secondary index is complicated because there could be multiple objects with the same secondary key and/or multiple versions (i.e., the most recent or obsolete ones) for the same object. Queries need to filter out the outdated version of each object. Moreover, the previous studies [11, 58] require point lookup on the primary index during the maintenance of the secondary index. Thus, to simplify the presentation of the secondary index, we only highlight the maintenance of the LSM R-tree. To handle efficient updates and support read queries in the LSM R-tree index, two update mainte-

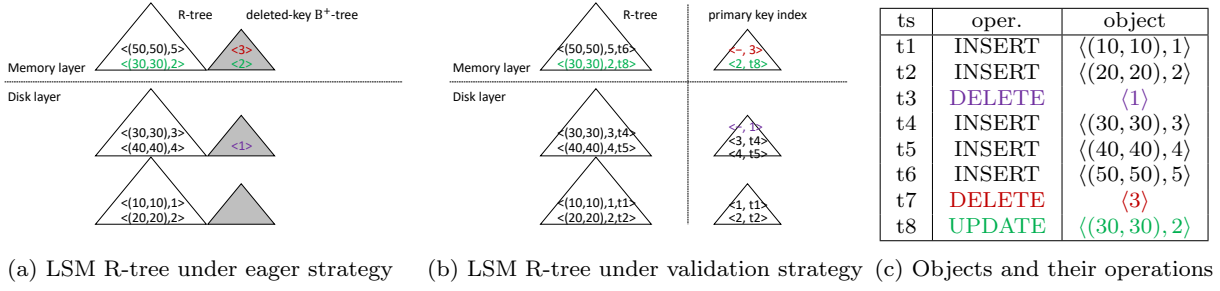


Fig. 1: Running examples

nance strategies, termed Eager [11] and Validation [58], have been proposed.

2.3.1 Eager Strategy for LSM Secondary Indexes

Alsubaiee et al. [11] introduce to LSM an additional deleted-key B⁺-tree for each R-tree to make the LSM R-tree consistent. The deleted-key B⁺ tree stores the primary keys (the object ids) of the deleted/updated objects to validate the state of an object given a query. The in-memory R-tree and its corresponding deleted-key B⁺-tree are tightly coupled, where they both make up the in-memory component of the LSM R-tree, and are flushed to disk together as a single component. Although the deleted-key B⁺-tree buffers the delete operations in the memory layer, the extra maintenance cost and disk I/Os during a search operation degrades the overall search performance.

Assume that we have an object, say $o = \langle Loc, O_{id}, \dots \rangle$, where Loc indicates the location of o , and O_{id} is o 's object identifier. Because the R-tree indexes locations, Loc is the secondary key for the R-tree index while O_{id} is a foreign key that points to the primary key of the object in the primary index, where the latter may contain other attributes that describe o . To insert o into the LSM R-tree, we add o into the in-memory R-tree, and do not need to access the deleted-key B⁺-tree.

To delete an entry $o = \langle Loc, O_{id} \rangle$ from the LSM R-tree, we perform the following steps: (1) Remove $o = \langle Loc, O_{id} \rangle$ from the in-memory R-tree index, if it exists, and (2) Invalidate the outdated os in the disk components by adding O_{id} into the deleted-key B⁺-tree in memory. Step 1 is necessary in the *Eager* strategy for the consistency of the LSM R-tree. For example, given a sequence of updates for an object, without performing Step 1 in the memory component, i.e., insert: $\langle Loc_1, O_i \rangle$, delete: O_i , and insert: $\langle Loc_2, O_i \rangle$, there is no way to identify which of the entries $\langle Loc_1, O_i \rangle$ and $\langle Loc_2, O_i \rangle$ is the recent location. As discussed by Alsubaiee et al. [11], entries in the deleted-key B⁺-tree help validate objects in the disk components.

To update a spatial object o inside the LSM R-tree, e.g., due to O_{id} 's changing its location from Loc_{old} to Loc_{new} , we perform the following: Delete the old object $o = \langle Loc_{old}, O_{id} \rangle$ (by following the delete procedure above), and then insert the new object $o = \langle Loc_{new}, O_{id} \rangle$ into the LSM R-tree (by following the insert procedure above).

To search the LSM R-tree (e.g., find all objects around $Loc = (2, 2)$), all the R-trees, whether in memory or in disk, are searched to find candidate query results because all the R-trees could have qualifying objects. Note that the secondary index, i.e., the R-tree, could have multiple records for a given secondary key, i.e., Loc , so that the results for searching with the secondary key could be in any component. Then, the O_{id} of each candidate will need to be searched against *all* the deleted-key B⁺-trees, and will be reported as output only if O_{id} does not exist in any of the deleted-key B⁺-trees of a newer component than the candidate's component. Notice that due to flushing the in-memory deleted-key B⁺-tree into disk along with its corresponding R-tree, there are multiple deleted-key B⁺-trees, and we start a new one in memory along with its corresponding new in-memory R-tree.

Running Example: Refer to Table 1c and Figure 1a for illustration. Assume that insert and delete events arrive in the order as indicated in the table, where each object is denoted by $\langle Loc, O_{id} \rangle$. Figure 1a illustrates how the *Eager* strategy handles inserts, deletes, and updates in the LSM R-tree. Each insert operation adds an object into the R-tree. For simplicity of illustration, let the R-tree be of Capacity 2, i.e., when an R-tree has two objects, it will be flushed to disk. By Time t3, Objects 1 and 2 will have been already inserted into the in-memory R-tree, and because its capacity is reached, the in-memory R-tree will be flushed to disk along with its corresponding empty deleted-key B⁺-tree (Both appearing in the figure in the bottom row). At Time t3, DELETE $\langle 1 \rangle$ makes the existing object $\langle (10, 10), 1 \rangle$ obsolete. Thus, $O_{id} \langle 1 \rangle$ is inserted into the deleted-key B⁺-tree (Appearing in Row 2 of the figure). Thus, when

searching the R-tree at any future time and returning object $\langle(10, 10), 1\rangle$, this output will be invalidated by the key $\langle 1\rangle$ that exists in the deleted-key B⁺-tree. Observe that the *Eager* strategy requires extra overhead to maintain multiple deleted-key B⁺-trees.

2.3.2 Validation Strategy for LSM Secondary Index

The Validation strategy [58] addresses the update overhead of the Eager strategy. It avoids using the deleted-key B⁺-tree and uses the *primary key index* to validate the query results. The validation strategy adds a timestamp in each object in both the *primary key* and *secondary* indexes. Figure 1b illustrates how the validation strategy handles data operations from the same running example above. For inserts, it inserts an object as $\langle Loc, O_{id}, ts \rangle$ into the R-tree where ts is a timestamp for the insert using local wall-clock time. No extra work is needed. For the delete at Time t_3 , the Validation strategy only inserts a control entry into the primary key index (e.g., $\langle 1, t_3 \rangle$, to indicate that Object 1 is deleted). For the update at Time t_8 , the tuple $\langle(30, 30), 2, t_8\rangle$ is inserted into the R-tree and the control entry $\langle 2, t_8 \rangle$ is inserted into the primary key index. Thus, it simplifies the delete or update procedures over the *Eager* strategy without searching the primary index to locate the object’s previous location and without searching the in-memory R-tree to remove the old entry. However, the *Validation* strategy’s query performance suffers as the validation steps uses the primary index for direct validation or primary key index for timestamp validation.

To summarize, the *Eager* and *Validation* strategies have inefficient update and search performance. For updates, the *Eager* strategy removes the old object entry by traversing the in-memory R-tree and inserting a control entry into the deleted-key B⁺-tree, and then inserts the new object, resulting in degraded update performance. The *Validation* strategy does not modify the R-tree upon update, but rather inserts the new object into the *primary key index* and the R-tree with a timestamp. In contrast to the *Eager* strategy, updates are faster in the *Validation* strategy. However, search in the *Validation* strategy degrades because the secondary index has false-positive results, and needs to validate the results by comparing against the primary key index.

2.4 The RUM-Tree: The R-tree with Update Memo

The Update Memo is an in-memory structure that helps detect and clean obsolete entries for secondary indexes.

The RUM-tree [75,82] handles update-intensive spatial workloads by augmenting an R-tree with an in-memory Update Memo structure (UM, for short). The

RUM-tree is a disk-based R-tree index that has a UM in memory. It maintains a global timestamp counter, and marks each object to show a temporal relationship among the objects. The UM maps primary keys to their update history in the corresponding secondary index. Each UM entry is of the form: $\langle O_{id}, ts, cnt \rangle$, where ts is the timestamp of the most-recent update to O_{id} in the RUM-tree, and cnt is the number of obsolete versions of Object O_{id} in the index. When an object, say o , is updated in the RUM-tree, it fetches and increments the RUM-tree’s global timestamp, say t , and inserts $\langle o, t \rangle$ into the RUM-tree. If a UM entry exists for o , its cnt increases by 1, and its ts updates to t . Else, a new UM entry with $cnt = 1$ and the new timestamp t is inserted into the UM. For deletes, only the UM is updated similarly as in update. If a UM entry does not exist for the deleted object o , a new entry in the UM is created for $\langle o, t \rangle$, and hence invalidating the most recent value of the object in the index wherever it is. A read operation searches the RUM-tree based on the search predicate as in a regular R-tree and returns a superset of the results. Then, each object in this superset is checked against the UM by comparing their ts against the ts in the UM indexed according to the O_{id} . If the ts in the object matches the ts in the UM, the object is determined as the current version and is returned as result. Otherwise, the object is obsolete and is discarded.

By handling the update operations in UM, the RUM-tree achieves significantly lower update cost without having big penalty during search. The UM has several cleaning strategies for removing the obsolete entries from both the R-Tree and UM to restrict the latter’s size. The RUM-tree shows improved performance on updates over the traditional R-tree [22,75,82]. However, it is not possible to directly use the RUM-tree as is in an LSM environment as the RUM-tree’s UM is tuned for use with the traditional R-tree, and if used as is, its UM maintenance would result in a large UM size without proper modification of its mechanisms. In this paper, we expand on the idea the UM, and propose new strategies to harmonize its use with an LSM-based R-tree secondary index. In the next section, we illustrate how we adapt UM for this purpose. We present the LSM-based R-tree’s insert, delete, update, and search algorithms, and demonstrate how they use the UM while controlling its size, and hence can always fit in memory.

3 The LSM RUM-TREE

We introduce the new LSM RUM-tree; an LSM R-tree augmented with an Update Memo (UM). The goal of the LSM RUM-tree is to efficiently handle updates, and improve search performance. From Section 2.3, existing

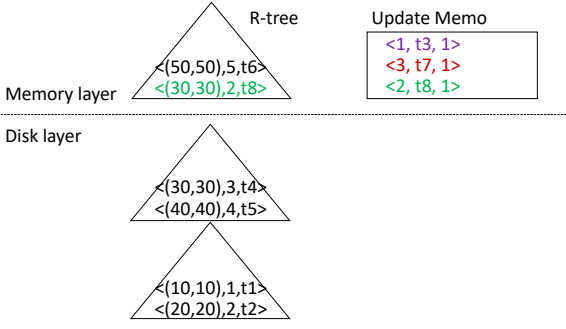


Fig. 2: LSM RUM-tree Overview

strategies for LSM secondary indexes degrade update and search performance. In the LSM RUM-tree, we introduce a UM within the LSM-based R-tree to simplify the processing of deletes and updates in the memory layer, and make disk-based search cost-efficient.

3.1 The Update Memo Structure (UM)

Refer to Figure 2 for illustration. In the LSM RUM-tree, the UM is based on a memory-resident hash map. The key to UM is an object identifier, and the value is a pair of recent timestamp and obsolete-objects counter. The UM has several roles. It keeps track of the deletes and updates ingested into the LSM RUM-tree. Because it resides only in the memory layer, the update to a UM entry takes constant time. Also, UM validates candidate objects resulting from a search operation. As in Section 2.3, previous strategies need to perform extra work to answer a query correctly. The *Validation* strategy validates output candidates of a search by comparing the output candidates against the *primary key index* in a way similar to that of the *Eager* strategy. Because UM is light-weight and memory resident, it enhances search performance over the previous approaches.

In Figure 2, each object in the LSM RUM-tree is represented as a triplet $\langle Loc, O_{id}, ts \rangle$, where Loc is the object's location, O_{id} is the object identifier, and ts is a global timestamp counter – an integer value incremented by 1 for each insert, delete, or update operation. The higher the value of ts , the fresher the object in the index. An entry e in UM is represented by a triplet $\langle O_{id}, ts, cnt \rangle$, where cnt is the number of outdated objects with the same O_{id} in the LSM RUM-tree. By default, the entry $e = \langle O_{id}, ts, 1 \rangle$ is first inserted into UM when deleting or updating an object with no existing entry in UM. Else, if O_{id} exists in UM in an entry, say e , from a previous delete or update, we increment $e.cnt$ by 1 and update its ts . When an obsolete object is found

and is removed from the LSM RUM-tree, we decrement the object's $e.cnt$ in UM by 1. Entry e is removed from UM when $e.cnt = 0$, i.e., there are no obsolete entries for O_{id} in the LSM RUM-tree.

3.2 Lazy Maintenance in UM

UM buffers deletes and updates in memory. Below, we discuss how the delete, update, and search operations in the LSM RUM-tree utilize UM.

3.2.1 Insert

Algorithm 1: Insert operation

input: Loc : Location (secondary key) of the object
 O_{id} : Object id (primary key)

- 1 $ts \leftarrow \text{timestamp counter}++$;
- 2 $O_{new} = \langle Loc, O_{id}, ts \rangle$;
- 3 $insertLSMRtree(O_{new})$;

Refer to Algorithm 1. During an insert, first we increment the global timestamp counter by 1 (Line 1). Note that the timestamp counter is an integer that gets incremented with each insert, delete, or update operation. Once we read the global timestamp counter, say ts , a new object $o_{new} = \langle Loc, O_{id}, ts \rangle$ is added to the in-memory R-tree (Lines 2-3). UM tracks only object deletes and updates, and performs no special actions for inserts. The goal of the LSM RUM-tree is to support both update- and insert-intensive spatial workloads. If UM treats inserts as in the case of the original RUM-tree [75,82], the UM size grows linearly with the number of objects for insert-intensive workloads. This would result in significant memory-space overhead. In the LSM RUM-tree, avoiding to maintain UM upon object inserts prevents the unnecessary growth in UM's size. In Section 3.3, we discuss in detail how we validate an object that has no entry in UM.

3.2.2 Delete

In the LSM RUM-tree, we minimize the processing needed to validate results. To delete an object, say o , we only add or modify o 's corresponding UM entry. As in Algorithm 2, if there is a UM entry, say e , for a given O_{id} , we set Field $e.ts$ to the global timestamp (Line 3) and increment $e.cnt$ by 1 (Line 4). If there is no such e in UM, we insert $\langle O_{id}, ts, 1 \rangle$ into UM. Note that $e.cnt$ corresponds to the number of obsolete copies of a given O_{id} in the R-trees. Naturally, each delete operation makes one additional obsolete object in the R-tree.

Algorithm 2: Delete operation

input: O_{id} : Object id (primary key)
1 $ts \leftarrow \text{timestamp counter}++$;
2 **if** entry e for O_{id} exist in UM **then**
3 $e.ts \leftarrow ts$;
4 $e.cnt++$;
5 **else**
6 put $e_{new} = \langle O_{id}, ts, 1 \rangle$ into UM ;

By tracking ts for freshness and cnt for the number of obsolete objects of O_{id} , we not only have a clear sense of a given object copy in the index whether it is fresh or not, but also have a good grasp of the number of obsolete copies of the object in the LSM RUM-tree. In Figure 2, “t3 : DELETE {1}” and “t7 : DELETE {3}” add $\langle 1, t3, 1 \rangle$ and $\langle 3, t7, 1 \rangle$ into the UM, respectively.

3.2.3 Update

To process an update, we check whether or not UM contains an entry, say e , with the same O_{id} . If e exists in UM , we update $e.ts$ to the current timestamp ts and increment $e.cnt$ by 1. If e does not exist, we add a new entry $e_{new} = \langle O_{id}, ts, 1 \rangle$ into UM. Then, we add the new object entry $\langle Loc, O_{id}, ts \rangle$ into the in-memory R-tree. Note that update consists of both delete and insert. Thus, we follow Lines 1-6 of Algorithm 2 and Lines 2-3 of Algorithm 1. In the example in Figure 2, “t8 : UPDATE $\langle (30, 30), 2 \rangle$ ” adds the entry $\langle 2, t8, 1 \rangle$ into the UM and inserts the new object $\langle (30, 30), 2, t8 \rangle$ into the in-memory R-tree. Notice that if there is another UPDATE for the same object at t9, we update the UM entry from $\langle 2, t8, 1 \rangle$ to $\langle 2, t9, 2 \rangle$ and insert the new object with the timestamp t9 into the R-tree.

3.3 Search

Algorithm 3: Validation with Update Memo

input : $candidates$: The list of candidates for a search
output: $results$: The list of results of a search
1 **for** $O_{cand} \leftarrow candidates$ **do**
2 $O_{id} \leftarrow O_{cand}.O_{id}$;
3 **if** entry e for O_{id} exist in UM **then**
4 **if** $O_{cand}.ts == e.ts$ **then**
5 $results.insert(O_{cand})$;
6 **else**
7 $results.insert(O_{cand})$;
8 **return** $results$

We explain how the LSM RUM-tree performs a search operation. Both the *Eager* and *Validation* strategies take extra disk I/Os to validate search results due to accessing the additional structures, either the deleted-key B⁺-tree or the primary key index (B⁺-tree). The UM is memory-resident (due to the cleaning strategies in Section 4). Thus, the LSM RUM-tree does not require a disk I/O except for accessing the disk-based LSM R-trees. By removing the additional tree structures from the LSM RUM-tree, we save on extra tree traversal times as well as on maintenance costs and expect to perform better during search.

Algorithm 3 illustrates how to utilize the UM to validate the search results returned from the LSM RUM-tree. First, we check whether a candidate O_{cand} from the R-tree search is fresh or not. If UM does not contain an entry with the same O_{id} , then O_{cand} is fresh, and is part of the search results. If there is an entry, say e , with the same O_{id} , then $O_{cand}.ts$ field is compared with $e.ts$ from UM. If $O_{cand}.ts < e.ts$, then the candidate is obsolete, and is discarded, else, is reported. Notice that there is no case where $O_{cand}.ts > e.ts$ because we always maintain a UM entry to reflect the most recent timestamp of an object. Also, if there is a fresh object (e.g., “t5 : INSERT $\langle (40, 40), 4 \rangle$ ” in Table 1c) and there is no other updates on the same object, there is no UM entry with the same $O_{id} = 4$. In Figure 2, if there is a range search from (30,30) to (40,40), the candidates in the LSM RUM-tree are $\{ \langle (30, 30), 3, t4 \rangle, \langle (40, 40), 4, t5 \rangle, \langle (30, 30), 2, t8 \rangle \}$. $\langle (30, 30), 3, t4 \rangle$ has smaller timestamp than its entry in UM ($\langle 3, t7, 1 \rangle$) because of the DELETE {3} at t7. Thus, this candidate is discarded. For $\langle (40, 40), 4, t5 \rangle$, UM does not contain an entry with $o_{id} = 4$, i.e., this object is fresh, and is in the search results. For $\langle (30, 30), 2, t8 \rangle$, its entry $\langle 2, t8, 1 \rangle$ shows the object is a part of the results as well. Thus, the final search results are $\{ \langle (40, 40), 4, t5 \rangle, \langle (30, 30), 2, t8 \rangle \}$.

Because UM is a hash map, massive amounts of deletes/updates will increase UM size. Also, as the hash map size increases, its lookup performance deteriorates due to the large number of entries in each hash map bucket and these result in degrading the search performance. For these reasons, in next section, we introduce LSM-aware UM cleaning strategies to bound the size of UM and improve search performance.

3.4 Comparison with Regular R-tree and RUM-tree

As discussed in the background section, the traditional disk-based R-tree [16, 35, 46] and RUM-tree [75, 82] maintain a single R-tree in memory and on disk. When an object gets deleted or updated (needing to delete the old object), an R-tree leaf node needs to be loaded

into DRAM, is modified, and is eventually written back to disk. While the RUM-tree can buffer the updates/deletes in the Update Memo, eventually it needs to apply similar delete operations in batch to the R-tree leaf nodes. Here, we discuss LSM RUM-tree's operations and exhibit how they improve performance than traditional R-trees.

The LSM RUM-tree supports four operations of concern: inserts, deletes, updates, and searches. For inserts, the LSM RUM-tree is write-optimized as it inserts new objects into the R-tree in the memory component. When the size of this in-memory R-tree reaches a certain size threshold, the in-memory R-tree is written to disk as a new component instead of updating existing R-trees on disk. Thus, inserts into the LSM RUM-tree should perform better than inserting into the regular disk-based R-tree that would incur random I/Os. For deletes, the LSM RUM-tree buffers the delete in the UM, and solely handles them as an update memo operation (in this case, the in-memory R-tree is not touched). So, both the LSM RUM-tree and the regular RUM-tree should perform the same in the case of deletes, and are both better than the delete operation in the regular R-tree. For updates, both the LSM RUM-tree and the RUM-tree should perform much better than the regular R-tree due to transforming the update into a delete operation touching the in-memory UM only, and an insert operation into the in-memory R-tree in the case of the LSM RUM-tree. Thus, the update in the LSM RUM-tree should perform better than the update in the RUM-tree, and both the LSM RUM-tree and the RUM-tree should perform better than the regular disk-based R-tree. For search, this is where both the regular R-tree and the RUM-tree outperform the LSM RUM-tree, which is expected when comparing, e.g., regular LSM search vs. regular B-tree search. Also, the RUM-tree search [75] performs slightly worse than the regular R-tree (due to the additional filtering step to exclude the obsolete objects from the answer).

4 LSM-Aware UM Cleaning Strategies

To ensure that UM fits in memory and remove obsolete entries from the tree, we control its size using cleaning strategies. We introduce various cleaning strategies that reduce the size of the UM efficiently without much overhead. As in Section 3.1, an entry e in the UM will be removed when the field cnt hits 0. Thus, our focus is on how to remove obsolete entries and decrease $e.cnt$ for each operation running on the LSM RUM-tree. We introduce four cleaning strategies for the LSM RUM-tree: (1) Buffered Cleaning, (2) Vacuum Cleaning, (3) Clean Upon Flush, and (4) Clean Upon Merge.

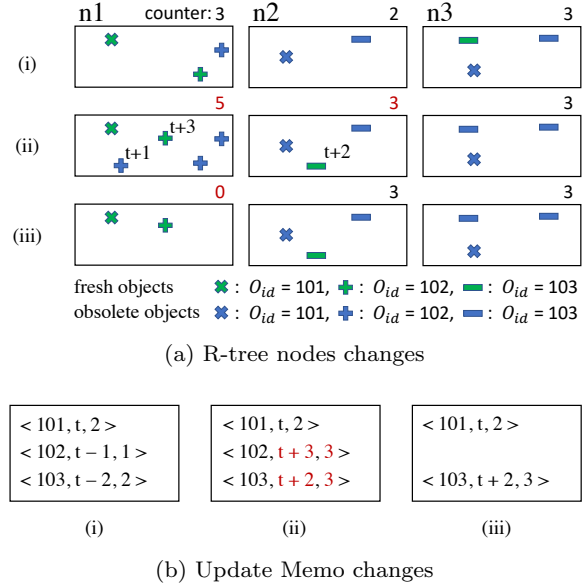


Fig. 3: Examples of Buffered Cleaning

The first two are for UM cleaning through the in-memory R-tree. In contrast, the remaining two are for UM cleaning through the disk-side R-trees. Although the RUM-tree [75] also employs cleaning strategies to clean the obsolete objects from the underlying R-tree, it is not straightforward to apply the same strategies directly into the LSM-based R-tree due to the added complexity of the LSM-based R-tree.

4.1 Buffered Cleaning

When an application uses the LSM RUM-tree as a secondary index to handle continuous update-intensive workloads (e.g., tracking moving objects continuously), there is a high chance that a node of the in-memory R-tree has multiple obsolete objects. To clean UM and those in-memory R-tree nodes, we introduce the Buffered Cleaning strategy that cleans an in-memory R-tree node based on the accumulated updates inside this node. This is particularly applicable for hot-spot cleaning. We maintain an *update counter* for each node of the in-memory R-tree. When the LSM RUM-tree buffers an update operation, the update counter on the node is incremented by 1. Once the counter hits some threshold, we scan the node using the UM, remove its obsolete objects and clean the UM entry by decrementing its cnt value by 1 for each of the removed obsolete objects. When a new R-tree node is created (e.g., due to a node split) or when Buffered Cleaning cleans a specific node, we set the node's update counter to 0.

For example, Figures 3a and 3b illustrate how Buffered Cleaning works in the LSM RUM-tree. There are three distinct objects (with O_{id} s 101, 102, and 103) over three R-tree nodes ($n1$, $n2$, and $n3$). We color the fresh objects green and the obsolete ones blue. Assume that the threshold for the update counter is 5. Figures 3a(i) and 3b(i) give the state of the R-tree nodes and that of UM at some point t . Notice that the counters for Nodes $n1$, $n2$, and $n3$ are 3, 2, and 3, respectively. Three more updates at $t+1$, $t+2$, and $t+3$ take place, and this results in Figures 3a(ii) and 3b(ii), as in Section 3.2.3. Because $n1$'s counter hits the threshold, we clean $n1$ as well as the UM. The threshold for the update counter is a variable that decides the frequency of invoking the Buffered Cleaning strategy. The lower the threshold is set, the more frequent the Buffered Cleaning strategy is invoked. Notice that frequent Buffered Cleaning results in extra cost to clean a node and the UM. Thus, we need to tune the threshold depending on the characteristics of the workload. For example, if the incoming location updates are continuous and gradual (e.g., due to movements of objects in small displacements), the lower threshold works efficiently. If location updates are periodic and geographically random (e.g., social media), the higher threshold can avoid extra cleaning-overhead.

Algorithm 4: A node and the Update Memo cleaning

```

input : objects: the list of objects in a node
1 for  $O \leftarrow \text{objects}$  do
2    $O_{id} \leftarrow O.O_{id}$ ;
3   if entry  $e$  for  $O_{id}$  exist in UM then
4     if  $O.ts < e.ts$  then
5       remove  $O$  from the node;
6        $e.cnt - -$ ;
7       if  $e.cnt == 0$  then
8         remove  $e$  from the Update Memo;

```

Algorithm 4 illustrates how we clean an R-tree node and its corresponding entries in the UM. While iterating over objects in a node, we discard the obsolete objects from the node by comparing the object's timestamp with the timestamp in the object's entry in the UM (Lines 2-5). If an obsolete object is removed, we decrement by 1 the object's corresponding $e.cnt$ entry in UM to track the number of obsolete objects with the same O_{id} (Line 6). If $e.cnt = 0$, there are no obsolete objects having the same O_{id} in the LSM RUM-tree. Thus, we remove e from the UM. Note that there are two cases that have $e.cnt = 0$: (1) There is only one copy of the object left having O_{id} in the LSM RUM-

tree and the object is fresh, or (2) There is no such object because all obsolete objects are removed, and the last operation for the object with the same O_{id} is a delete operation. The Buffered Cleaning strategy handles both cases consistently. By the end of the cleaning steps, Buffered Cleaning sets the update counter to 0.

4.2 Vacuum Cleaning

Vacuum Cleaning complements Buffered Cleaning because the latter targets mostly the hot-spot nodes in the LSM RUM-tree. There are still some cases that Buffered Cleaning cannot handle very well: (1) A node is on a cold-spot so the counter for the Buffered Cleaning does not hit the threshold or (2) Objects in a node have been obsoleted because of updates in other nodes. Not cleaning these cold-spot nodes can result in growing the size of UM and being not able to control it. To make up for the cold-spot nodes not handled by Buffered Cleaning, we introduce Vacuum Cleaning for fair cleaning of in-memory R-tree nodes to bound UM's size. The goal of Vacuum Cleaning is to iteratively scan the in-memory R-tree, and remove obsolete entries.

Vacuum Cleaning is triggered after the in-memory R-tree has ingested a certain amount of updates, and it picks an R-tree leaf node to clean. In Vacuum Cleaning, we maintain a global counter and a vacuum cleaner. The global counter stores the number of updates in the entire LSM RUM-tree, and the vacuum cleaner holds the next leaf node to be cleaned in the in-memory R-tree. Once the global counter hits some threshold by update operations, we clean the "next node" by the vacuum cleaner. The node and UM cleaning are the same as the ones in Algorithm 4. After this node's cleaning is finished, we reset the global counter to 0 and set the vacuum cleaner to the next leaf node. We can skip the nodes that have been recently cleaned by the Buffered Cleaning Strategy. We can adjust the frequency of Vacuum Cleaning by varying the threshold th of the global counter. If the number of leaf nodes of the in-memory R-tree is N , it takes at most $N \times th$ updates to fully clean the R-tree. For example, small th leads to a shorter *full cycle* as in Figure 4. Depending on the capacity of the R-tree, tuning th is needed for best performance results. Similar to Buffered Cleaning, a lower value of th results in frequent Vacuum Cleaning executions, and may lead to excessive overhead. Depending on the size of the in-memory R-tree and its node capacity, a fine tune will be needed to clean the R-tree efficiently and avoid extra overheads.

Discussion: The Buffered and Vacuum Cleaning strategies have several advantages. They help reduce the UM size. Having obsolete objects in the in-memory

R-tree leads to unnecessary UM entries. By cleaning the UM, we bound its size. Also, both cleaning strategies clean the nodes of the in-memory R-tree, and hence reduce its size. Update-intensive workloads cause many obsolete objects in an R-tree node, and this leads to unnecessary R-tree node splits and R-tree flushes to disk. By cleaning the nodes in the in-memory R-tree, the nodes remain fresh, and avoid unnecessary splits due to being filled with obsolete objects. Moreover, slots used by obsolete entries can be used to accommodate future inserts/updates, and underflow R-tree nodes may get merged to reduce the R-tree size. Therefore, the in-memory R-tree is less likely to hit the LSM-tree memory component threshold, and reduce the amount of flushes (disk I/Os). Also, by cleaning R-tree nodes, we expect to improve search performance because there should be less false-positive candidates (the obsolete entries in a leaf node) while searching the R-tree. Notice that the Buffered and Vacuum Cleaning strategies perform in-place updates only for the main memory component. Therefore, there is no additional (random) I/O required. Moreover, it cleans the in-memory R-tree leaf node either when a write operation reaches the leaf node, or through the next leaf node pointer held globally maintained by the vacuum cleaning process. There is no additional overhead to locate the in-memory node for cleaning. To the contrary, the *Eager* strategy [11] does not have such cleaning mechanism, and requires searching and removing the obsolete entry in the in-memory R-tree on each delete/update which has overhead.

4.3 Clean Upon Flush

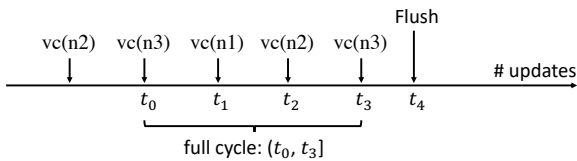


Fig. 4: Cleaning cycle by Vacuum Cleaning

The Buffered and Vacuum Cleaning strategies are for cleaning the UM and the in-memory R-tree. For LSM-based indexes, there are unavoidable times when these two approaches miss obsolete objects, and thus the UM size increases, e.g., when LSM decides to dump the memory contents to disk while the in-memory R-tree is not fully cleaned. We introduce the Clean Upon Flush cleaning technique that will be executed during

the flush operation of the LSM RUM-tree. Assume that there are three nodes in the in-memory R-tree, that Vacuum Cleaning runs on the R-tree nodes in a round-robin manner, and that Buffered Cleaning is off. In the LSM RUM-tree with component-based flush, a component is flushed to disk when the size of the memory component exceeds its capacity. In Figure 4, periodic Vacuum Cleaning (i.e., $vc(node)$) takes place as updates accumulate in the LSM RUM-tree, and there is a flush operation at t_4 . Note that even though it has completed a full vacuum cleaning cycle at t_3 , there should be obsolete objects at the flush time t_4 . Precisely, at the end of the last vacuum cleaning cycle (t_3), all obsolete objects made before t_1 are cleaned. However, some obsolete objects (e.g., ones made between t_1 and t_3 in Node $n1$) are not cleaned at t_4 because the last clean on $n1$ was at $t1$. Also, all obsolete objects after t_3 are not cleaned. Once the in-memory component is flushed into disk, it is expensive to modify the disk component because it will lead to extra disk I/O, and will even violate the LSM policy: the disk components of the LSM-tree are immutable.

Thus, we introduce the Clean Upon Flush cleaning strategy that cleans the obsolete objects before they are moved to disk. In Clean Upon Flush, we bring the flushed component up-to-date at the time of flushing without having too much computational overhead. Mainly, we add the R-tree and UM cleaning step just before flushing. When the LSM RUM-tree is flushed into disk, it orders the objects by the Z-order or Hilbert curves [38, 65] to be stored in disk efficiently. For Clean Upon Flush, when the flush operation orders the objects, we check whether the object is obsolete or not by comparing it with the entry in the UM as in Algorithm 4. If the object is obsolete, we discard the object so it does not get flushed into disk, and we clean the corresponding UM entry. If the *cnt* of the UM entry is 0, we remove the entry from UM.

Discussion: Clean Upon Flush is expected to improve performance in various ways. First, the UM size is reduced due to cleaning. Shrinking the UM size is a top priority in all cleaning strategies. Furthermore, all the objects in the flushed component are fresh at the time of the flushing. Thus, the size of the flushed component could be shrunk dramatically in some cases. For example, if most objects in the in-memory R-tree are obsolete and only a few objects are fresh, the size of the flushed R-tree will be proportional to the number of fresh objects. Consider an extreme case of having million updates from a hundred distinct objects. Following a Clean Upon Flush, only hundred fresh objects will be presented in the flushed R-tree in contrast to million. Thus, while *Eager* and *Validation* strategies may incur

the same amount of flushes as Clean Upon Flush, the amount of blocks and disk I/Os may be drastically different, and searching the flushed LSM RUM-trees will require fewer disk I/Os due to the smaller flushed components. As a result, flush latency and search performance would be improved significantly. To the contrary, the *Eager* strategy has no obsolete entries in memory [11] but its delete B-tree may contain tuples for the disk components incurring overhead for flush operations. The Validation strategy does not clean obsolete entries upon flush [58]. Therefore, the flush component may contain obsolete entries that increase the flush size and the number of I/Os.

4.4 Clean Upon Merge

Despite having in-memory cleaning strategies and cleaning at flush time, we can still have obsolete objects in disk. For example, if there is a fresh object O_{disk} having some O_{id} somewhere in the disk R-tree, any update to O_{id} will make O_{disk} obsolete. This is likely to happen in moving-objects applications as the objects' locations are updated frequently. **Thus, the objects in disk components become obsolete as new updates accumulate into the in-memory component.**

We introduce the Clean Upon Merge cleaning strategy to clean the obsolete objects on disk as well as their corresponding UM entries. Direct modifications on the disk component are costly and not suitable for LSM. Thus, we leverage the LSM merge operation. The target of the LSM merge operation is to bound the number of disk components via merging, and hence control the search time. Having many disk components degrades search performance as the search needs to investigate all disk-based R-trees. For the merge operation, multiple disk components are bulk-loaded and are merged into a single disk component. With Clean Upon Merge, we have a validation step as the one in Algorithm 4 on the bulk-loading procedures of the merge operation. When bulk-loading the existing components, we check each object on-the-fly against the UM entries. If an object is obsolete, we discard it. Obviously, we update the UM entries accordingly in a way similar to Clean Upon Flush and the other cleaning strategies.

Discussion: Clean Upon Merge cleans the obsolete objects in the disk components with minor modifications to the LSM merge operation. Notice that it is guaranteed that there will be no obsolete objects (i.e., an empty UM) in a system when there is only one disk R-tree after the merge with Clean Upon Merge. If combined with Clean Upon Flush, both can enhance search performance due to accessing fewer and more compact R-trees on disk. Experimental results show

that the query performance with Clean Upon Flush and Clean Upon Merge achieves over 400x speedup when compared to the Validation strategy. Observe that the LSM RUM-tree's UM is fully in-memory that requires no additional I/Os for validation. The *Eager* [11] strategy's merge cleaning requires checking all delete B-tree of newer disk components. The Validation [58] strategy contains obsolete entries when flushing an in-memory R-tree. The larger component may incur more I/Os during the merge operation. It has to check the primary key index (an LSM B⁺-tree) components that have newer entries determined by timestamps. The extra disk I/Os create additional overheads in the merge cleaning by the *Eager* and *Validation* strategies in contrast to the LSM RUM-tree.

5 Concurrency Control

The memory component of the LSM RUM-tree with a latch protocol protects a node in the tree that performs concurrent read/write operations in a multithreaded environment. When traversing the LSM RUM-tree, a thread is required to acquire/release latches on the nodes for a safe search or insert/update operations. For example, AsterixDB follows the latch protocols described in ARIES/KVL [62] and GiST [50] for the B-tree and the R-tree, respectively. Although a traditional LSM-tree protects its memory component by the latch protocols, introducing the Update Memo poses new challenges to support concurrent operations over UM because a corrupted data structure (e.g., incorrect ts or cnt of an UM entry) would lead to incorrect search results or UM cleaning failures. In this section, we demonstrate how to implement and achieve concurrency in UM. We implement LSM RUM-tree on top of existing LSM R-tree implementations in AsterixDB [1, 10, 11, 58] with existing concurrency control protocol. Thus reads and writes in LSM RUM-tree are protected by the its original latching protocol. It guarantees concurrent access and modification of the in-memory R-tree, and thread-safe flushes and merges. That leaves us the last task of ensuring the correctness of concurrent operations related to the Update Memo.

5.1 Concurrency Control in Update Memo

We present how the Update Memo can support concurrent operations in a multithreaded environment. As discussed in Section 3.1, UM is based on a hash map structure where the key to UM is an object identifier (O_{id}), and the value is a pair of recent timestamp (ts)

and obsolete object counter (cnt). When an existing object is deleted or updated, it becomes an obsolete object in the LSM tree, so we update ts to the current timestamp and increment cnt for the UM entry of O_{id} . On the contrary, if we perform LSM-aware UM cleanings as in Section 4, we decrement cnt of O_{id} 's UM entry because the cleaning removes the obsolete objects.

Algorithm 5: Compare and If Less then Swap

input : O_{id} : Key of an object
 val : Timestamp to be compared

```

1  $ts \leftarrow UM[O_{id}].ts$ ;
2 repeat
3    $curr \leftarrow ts.get()$ ;
4   if  $curr \geq val$  then
5      $\perp$  return  $curr$ ;
6 until  $ts.compareAndSet(curr, val)$  is true;
7 return  $val$ ;
```

To support concurrency in UM, first, we need to make ts and cnt in the UM entry atomic integers so that any thread can read or write the value without conflict. In addition, we introduce a new atomic primitive, *Compare and If Less, then Swap* (CILS, for short) to update the timestamp value of an entry. The CILS atomic operation is a variation of the Compare and Swap (CAS) operation [37]. CAS compares the current value of a memory location with a given value, and then modifies it only if the two values are the same. However, CAS fails in our strategy when multiple threads are trying to update the same memory location of ts and the last thread that succeeds in CAS does not hold the highest value of ts . If concurrent threads th_1 with $ts = 4$ and th_2 with $ts = 5$ try to update the same memory location and th_1 succeeds later than th_2 , ts ends with 4, which is not the highest ts . The CILS(m, v_{old}, v_{new}) operation, as the name implies, compares v_{old} (i.e., the old value) at m (i.e., a memory location) with v_{new} (i.e., the new value), and modifies m only if v_{old} is less than v_{new} . The reason is that UM always needs to hold a value for the most recent object's timestamp that has to be the highest value (most recent timestamp) among the objects with the same key.

Algorithm 5 shows how CILS is implemented on UM. In Line 1, a thread gets the atomic integer variable (ts) that will be compared to a given val . After the thread gets its value ($curr$) atomically (Line 3), it is compared with val . If $curr$ is greater than or equal to val , that means another thread has already updated the timestamp with a larger value, so we abort the CILS operation (Lines 4-5). If $curr$ is less than val , the thread tries to update $curr$ to val (Line 6). If the CAS operation (in Line 6) fails (i.e., $ts.compareAndSet()$ is

false), then the thread repeats the operations until it succeeds. With CILS, we always keep the largest (the most recent) ts for a given object.

Algorithm 6: Concurrent update to the memo

input : O_{id} : Key of an object
 ts : Timestamp of updated object

```

1  $ret \leftarrow UM.putIfAbsent(O_{id}, (ts, 1))$ ;
2 if  $ret$  is not null then
3    $curr \leftarrow UM.CILS(O_{id}, ts)$ ;
4    $check \leftarrow UM.putIfAbsent(O_{id}, (curr, 1))$ ;
5   if  $check$  is not null then
6      $\perp$   $UM.atomicIncCnt(O_{id})$ ;
```

As discussed in Section 3.2, delete/update operations in the LSM RUM-tree require the addition or update of the UM entry. Algorithm 6 shows how the UM entry is added or updated during the insert/update operation. Right after an update/deletion of an object in the R-tree component, the algorithm is executed. In Line 1, we call `putIfAbsent()` on UM with the key O_{id} and the value $(ts, 1)$. This is for initializing the entry atomically if the given O_{id} does not exist in UM. If multiple threads execute the `putIfAbsent()` function at the same time, only one thread will get *null* as a return, and the other threads will get some mapping value (i.e., (ts, cnt)) for O_{id} . If the entry exists, ret is not *null*, and we execute the CILS operation as in Algorithm 5, then atomically increment cnt of the entry for O_{id} in UM (Lines 4-6).

For the UM cleaning policies discussed in Section 4, we decrement cnt of an UM entry and remove it if the value equals 0. During the cleaning operations, ts of an object O_{id} remains the same, but we only need to decrement cnt for the obsolete objects removed from the LSM RUM-tree (Algorithm 7).

Algorithm 7: Update Memo Clean

input : O_{id} : Key of an object

```

1  $ret \leftarrow UM.atomicDecCnt(O_{id})$ ;
2 if  $ret == 0$  then
3    $\perp$   $UM.remove(O_{id}, 0)$ ;
```

5.2 Implementation and Correctness

Because there is no hardware support (CPU instruction) for the CILS operation, we need to emulate it in high-level languages using a spin-lock approach with

CAS as in Algorithm 5. Next, we need to use a thread-safe UM data structure for concurrency. As in Algorithm 5-7, UM operations need to be thread-safe as multiple-threads read and/or write simultaneously. Since we implement LSM RUM-tree in AsterixDB [1,10] which is written in Java, we choose such data structures written in Java as well. *ConcurrentHashMap* [5] in JAVA, is an example for this purpose that supports thread-safe operations by having separate locks for different hash buckets for various HashMap operations, e.g., *put()*, *putIfAbsent()*, or *remove()*. In addition, Variables *ts* and *cnt* need to support atomic operations for incrementing and decrementing *cnt* and *compareAndSet()* on *ts* in the CILS operation.

To ensure that CILS and UM operations are thread-safe, we divide the operations into three cases that UM can handle, and we prove their correctness.

5.2.1 Concurrent Updates on UM Entries

Assume that $UM[O_{id}].ts$ (the current timestamp of an object) is **1** and $UM[O_{id}].cnt$ (the current number of obsolete objects for O_{id}) is **1**. Assume further that two threads, say th_1 and th_2 , update UM concurrently with timestamp values **6** and **7** as in Algorithm 6. After the execution of Line 1, both th_1 and th_2 get the same $ret = 1$. Then, they execute the CILS operations as in Algorithm 5. In Line 3, both threads have the same *curr*, which is equal to **1**. Thus, both the concurrent executions for th_1 and th_2 skip Lines 4 and 5 as *val* is **6** and **7** for th_1 and th_2 , respectively. Then, each thread executes *ts.compareAndSet(curr, val)* at Line 6. Because *ts* is an atomic variable and the *compareAndSet()* operation atomically sets *val* only if *curr* equals *val*, either th_1 or th_2 will succeed and exit from CILS. i) If th_1 succeeds, the current $UM[O_{id}].ts$ will be set to **6** and exit the CILS operation. At this point, th_2 will continue the loop again. Because *curr* is now set to **6** and *val* of th_2 is **7**, it skips Lines 4-5 and tries to execute *compareAndSet(curr, val)* again. After th_2 completes the operation, $UM[O_{id}].ts$ is set to **7**, which is the desired result of *ts* (i.e., the most recent timestamp of O_{id}). On the contrary, ii) if th_2 succeeds the *compareAndSet(curr, val)* first in the initial loop, then $UM[O_{id}].ts$ would be set to **7**. In this case, th_1 will exit the CILS in Line 5 of the second loop because $UM[O_{id}].ts = 7$ is already larger than *val* of th_1 , which is **6**. After the CILS operation, both threads execute an atomic increment of *cnt* for O_{id} and the final value will be **3** as it is incremented twice by th_1 and th_2 . Even if there are N interleaving threads concurrently accessing the same UM entity, regardless of the *ts* order, each thread increments *cnt*, and then updates $UM[O_{id}].ts$

only when the *ts* is less than a thread's *ts*. Therefore, this execution is what is expected to happen and would result in correct behavior of the concurrent updates.

5.2.2 Concurrent Cleaning of UM Entries

Assume that th_3 performs the Vacuum Cleaning on a leaf node of R-tree for O_{id} while th_4 executes the Clean-Up-on-Flush operation, which also involves O_{id} . The cleaning process reduces obsolete objects from LSM RUM-tree, and does not affect the *ts* of a UM entry. Thus, we only need to execute an atomic decrement of $UM[O_{id}].cnt$ for UM updates. Either one of the two threads th_3 or th_4 will get $ret = 0$. This will be the thread executed lastly because of the nature of *atomicDecCnt()*. Hence, only this last thread will remove the UM entry for O_{id} . There is an edge case where the removal of an UM entry could lead to a wrong result. We handle this case in the next section.

5.2.3 Concurrent Update and Clean Operations over UM Entries

Assume that there are two copies of the object O_{id} in LSM RUM-tree (one current and one obsolete), and both are placed in the memory component. Suppose that the values of *ts* are **9** and **7** for the recent and obsolete object, respectively. In this case, the current UM entry (i.e., $UM[O_{id}]$) is $\langle id, 9, 1 \rangle$. We need to check the correctness of the concurrency when multiple threads are involved in both UM update and cleaning operations (i.e., th_5 updates the UM entry at $ts = 10$ and th_6 executes Vacuum Cleaning to remove the obsolete object, concurrently).

i) If th_5 reaches Line 4 of Algorithm 6 earlier than Line 1 of Algorithm 7 of th_6 , it is straightforward that the UM entry will be set to $\langle id, 10, 2 \rangle$ by th_5 and then is updated to $\langle id, 10, 1 \rangle$ by th_6 .

Even ii) if th_5 executes Line 6 after th_6 's Lines 1-2, the UM entry changes to $\langle id, 10, 0 \rangle$ by th_6 , and then $\langle id, 10, 1 \rangle$ by th_5 because th_6 has not yet removed the UM entry (Line 3). After th_5 finishes the update, Line 3 of th_6 will be aborted because the current $O_{id}.cnt$ is not equal to 0, which is what is expected to happen.

Lastly, iii) if th_6 completes Lines 1-3 while th_5 executes Line 3, the UM entry for O_{id} has already been removed by th_6 . Therefore, th_5 continues to re-set the UM entry at Line 4. If there are multiple updates, only one thread will initialize the UM entry with *curr* and the others just increment *cnt* atomically (Lines 5-6). This is the correct and expected behavior that is consistent with i) and ii).

5.2.4 Concurrent Read

Assume that Thread th_7 executes a range search, and searches the in-memory and disk R-tree components. Notice that the global timestamp is ascending-only. We claim that when th_7 finds an obsolete object, say o , storing object ID, say oid , during search, it is guaranteed to see an UM entry e with a larger ts than the one th_7 finds in o . As discussed earlier, when an object becomes obsolete, an UM entry e is inserted/updated with a larger ts . e will only be removed if the LSM RUM-tree contains no obsolete entries with oid . Concurrent deletes and updates may update the $e.ts$ and $e.cnt$ entries, but will only increment those values. Therefore, th_7 can still determine that o is obsolete. In our implementation based on AsterixDB [10]’s LSM R-tree, latches are held at the node level, and UM’s Buffered, Vacuum, and Flush clean policies need to acquire exclusive latches to proceed. Therefore, no concurrent thread is able to clean o when th_7 is reading it, decrementing $e.cnt$, or removing it. Thus, we guarantee that the concurrent queries will be able to determine which objects are obsolete or current.

6 Performance Study

We evaluate the LSM RUM-tree along with its insert, delete, update, and search performance. All the experiments are conducted on multi-socket server with Intel(R) Xeon(R) Platinum 8168 CPU @ 2.70GHz processors that has 24 cores and 387GB DRAM per NUMA node. All experiments are run under a single NUMA node to eliminate NUMA effects. We use two real datasets, Gowalla [24] and ChicagoTaxi [66] with millions of points, and one synthetic data set: BerlinMOD [31], with sizes Gowalla (6.4m), ChicagoTaxi (15m), and BerlinMOD (56m). For the datasets, there are 107k, 5.2k, and 2k, and unique keys (Object IDs), respectively, along with their locations over time. Notice that most of the data entries in each dataset represent an update of an object’s location. Here, we emphasize the focus of the LSM RUM-tree to handle moving objects and location updates instead of simply being write optimized. All the datasets exhibit temporal localities in updates among the same object as the updates are sorted by timestamps.

We implement the LSM RUM-tree inside AsterixDB [1] in Java, and compare the LSM RUM-tree with the existing *Eager* and *Validation* strategies already implemented in AsterixDB [11, 58]. For the LSM RUM-tree, we set the budget of the in-memory R-tree to 256 MB and the page size to 2 KB following [11]. The merge policy is set to the *prefix* policy with Threshold=5. We

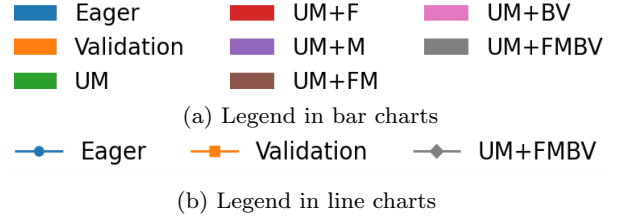


Fig. 5: Legends used in the experiments

augment the UM implementation into the already existing LSM R-tree. To study the effect of only UM, we do not use any optimization, e.g., a bloom filter [17] or range filter [12] on the LSM R-tree since they are orthogonal to the focus of this paper.

We use the following notation to refer to the various cleaning strategies. *UM* denotes LSM RUM-tree without any cleaning strategy, *UM+(cleaning strategies)* denotes LSM RUM-tree with combinations of cleaning strategies, where F, M, B, and V refer to Clean Upon **F**lush, Clean Upon **M**erge, **B**uffered Cleaning, and **V**acuum Cleaning, respectively.

As in Sections 4.1 and 4.2, we tune the threshold to get the best overall performance. Lowering the threshold cleans the UM entries and the in-memory tree nodes more frequently, but requires extra computation cost. We empirically set the threshold to 4 and 8 for the update counter of Buffered Cleaning and the global counter of Vacuum Cleaning, respectively. Figure 5 gives the legends we use throughout all experiments except for Figure 8 that uses its own legend. All bar charts use Figure 5a and all line charts use Figure 5b.

6.1 Comparison with Baselines

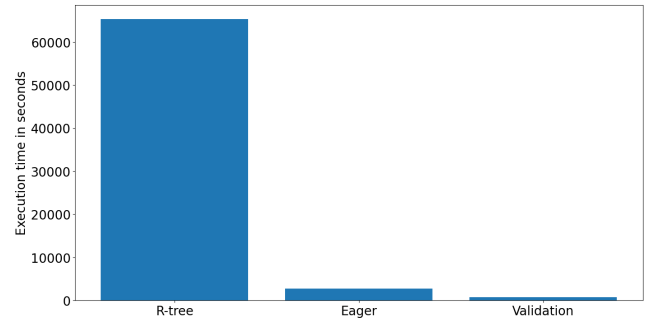


Fig. 6: Comparison of Update Performance Against Baseline Indexes

First, we evaluate existing baselines in AsterixDB [1, 10, 11, 58], namely the regular disk-based R-

tree [35], and the LSM R-tree with *Eager* [11] and *Validation* [58] strategies. In this section, we demonstrate the advantages and disadvantages of traditional R-tree against LSM R-tree, and compare LSM R-tree baseline strategies. In this experiment, we choose Chicago-Taxi [66] dataset as it has a relatively large size (15m data entries) with a moderate amount of unique objects. First, we ingest all updates of the dataset in each baseline index, and then execute range queries with different selectivities. The results for ingesting spatial objects and their location updates are given in Figure 6.

When processing location updates, R-tree exhibits orders of magnitude worse performance than both LSM R-tree strategies. The result is as expected. When updating location of an object o , R-tree requires deleting o 's old entry (that may reside in a leaf node on disk), and inserting a new entry to the R-tree leaf node (that may also reside on disk). Random I/Os are needed to load these R-tree nodes and write them back. LSM R-tree does not load any disk component in memory for updates or deletes (no additional I/Os), but create new versions in the in-memory R-tree as discussed in Section 2. Thus its disk writes are purely sequential. Even during a merge operation, disk components are loaded into memory, merged, and written back to disk sequentially. Therefore LSM R-tree exhibits much better performance. The regular R-tree takes over 18 hours to ingest the whole dataset, while the *Eager* and *Validation* strategies only take 46 minutes and 12 minutes, respectively.

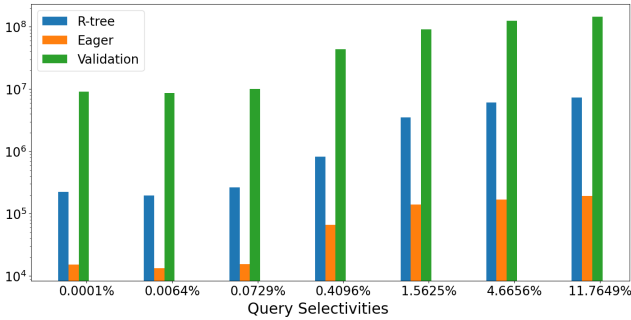


Fig. 7: Comparison of Query Performance Against Baseline Indexes

The query performance on each dataset is given in Figure 7. R-tree performs the second best in this experiment. R-tree removes obsolete entries in place for deletes or updates. Therefore it doesn't need to validate entries as in the *Validation* strategy and saves in validation overheads. R-tree performs around 1 order of magnitude worse than the *Eager* strategy. While *Eager* strategy doesn't update objects in place, it executes

merge operations as the disk components accumulate and remove obsolete entries. Therefore after ingesting all updates, the disk R-tree of LSM R-tree *Eager* strategy doesn't contain much obsolete entries, and store its disk components sequentially and compactly. Therefore it can provide purely sequential I/Os for the disk components, and achieves good performance.

Through the baseline experiments, we can see that LSM R-tree exhibits orders of magnitude better performance than the disk R-tree in both location updates and range queries. This justifies the need of LSM R-tree for high performance. Moreover, There is no clear winner between *Eager* and *Validation* strategies as one is more read-optimized and the other is more write-optimized. Our LSM RUM-tree aims to achieve superior performance in both read and write compared to those baseline LSM R-trees.

6.2 Update Performance

To represent various scenarios of update-intensive workloads, first, we sort the data objects (inserts or updates of locations) by their timestamps, then ingest the data objects into each LSM RUM-tree. The first ingestion of each data object is an *insert*, then is followed by multiple *update* operations. Therefore insert/update operations arrive in the data stream according to the real-world timestamps. We measure the total execution time to complete the data ingestion of all inserts and updates, excluding setup and data feeding time. When comparing the LSM RUM-tree against the LSM R-tree, to highlight only the performance of the LSM R-tree, we exclude the point lookup time on primary key in the *Eager* strategy of the LSM R-tree, and just feed the old location of an object along with the object delete and update requests.

Figure 8 gives the update performance on three datasets with respect to update procedures, the time to process a flush operation, and the time to process a merge operation by comparing the LSM RUM-tree mechanisms with the *Eager* and the *Validation* strategies of the LSM R-tree. As in Section 4, the LSM RUM-tree cleaning strategies clean obsolete objects in both the memory and disk layers as well as the UM entries. In update-intensive workloads, entries in the LSM R(UM)-tree become obsolete very frequently, and cleaning obsolete objects improves the update performance because it avoids unnecessary node splits in the R-tree, controls in-memory R-tree's memory footprint, minimizes the flush/merge operations, and saves in disk I/Os. Table 1 gives the total number of flush and merge operations for each dataset's updates. From the experiments, observe that the LSM RUM-tree with the all cleaning strategies

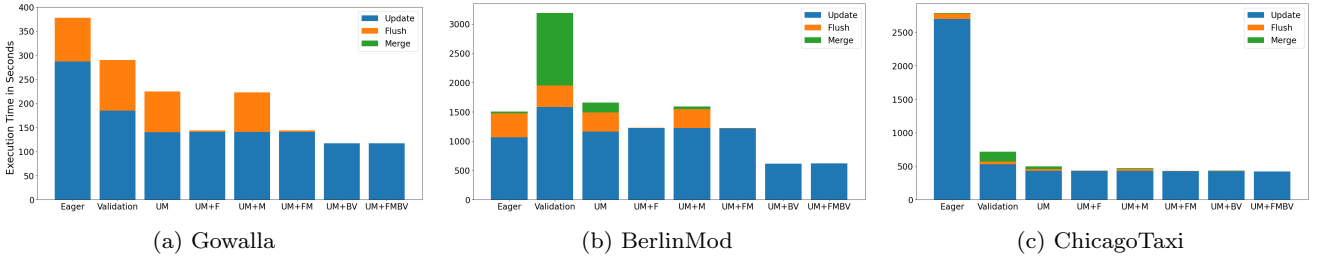


Fig. 8: Comparisons of Update Performance on LSM R-trees

achieves 2.4x to 6.6x speedups over the *Eager* strategy and 1.7x to 5.1x better than the *Validation* strategy depending on the different datasets. Even without any cleaning strategies, which is not the recommended typical use of the LSM RUM-tree, the LSM RUM-tree performs the best in Gowalla and ChicagoTaxi while being only slightly slower than *Eager* strategy in BerlinMod. This demonstrates that the buffered deletes and updates has the potential to improve performance in update-intensive workloads. The *Eager* strategy has to search the in-memory R-tree for each update, and inserts full object entry into the delete-key B-tree. But it performs well in the BerlinMod dataset because updates of moving objects have localities and are likely falling into the same R-tree nodes. Thus, in-place cleaning of the *Eager* strategy allows more space for further updates. The *Validation* strategy saves in searching the in-memory R-tree, but still needs to modify the more heavy weight primary key index than the LSM RUM-tree’s UM. It has large overhead in merge in BerlinMod. BerlinMod has a small set of unique objects and many updates. Thus, the *Validation* strategy creates a large amount of obsolete objects on disk due to not cleaning in place or during flush. The problem may be mitigated by implementing better merge repair policies but even without counting the merge time, the LSM RUM-tree performs around 2x better than the *Validation* strategy.

strategy	Gowalla		BerlinMOD		ChicagoTaxi	
	Flush	Merge	Flush	Merge	Flush	Merge
Eager	3	0	28	6	14	3
Validation	4	0	40	9	13	2
UM	3	0	30	6	10	2
UM+F	3	0	30	6	10	2
UM+M	3	0	30	7	10	2
UM+FM	3	0	30	7	10	2
UM+BV	0	0	0	0	6	1
UM+FMBV	0	0	0	0	6	1

Table 1: The number of Flush and Merge operations for each dataset

Next, we evaluate each cleaning strategy in detail. Notice that the expected cleaning effect of the Clean

Upon Flush (i.e., *UM+F*) among all datasets is that it does not reduce the number of flushes but reduces in the amount of I/Os. As it discards obsolete objects in the in-memory R-tree on a flush operation, it gets much smaller than the original uncleaned one. Thus, eventually, after cleaning, a fewer number of blocks of data is to be written into disk. As in Figure 8 and Table 1, the performance of *UM+F* reduces the processing time for the flush operation by up to 55x while performing the same number of flush operations. In the update-intensive workloads scenario, the in-memory R-tree would contain many obsolete objects. By filtering the obsolete objects in the flushing procedures, the size of the flushed component is shrunk and the update operation cost is reduced dramatically. In comparison, while the *Eager* strategy removes obsolete entries in the in-memory R-tree immediately, its delete-key B-tree may still contains entries for the obsolete objects on disk and needs to be flushed to disk, causing additional I/O overhead. Similarly, the *Validation* strategy leaves obsolete entries in the in-memory R-tree during flush, and has to flush the primary key index that requires more I/Os in each flush operation.

Figure 8b illustrates that Clean Upon Merge (i.e., *UM+M*) significantly reduces the time for the merge operations by a factor of 3 when compared to UM. Notice how efficiently the *UM+M* cleans the obsolete objects in the disk layer. While *UM* merges all objects in the disk layer without cleaning, the *UM+M* discards the obsolete objects using UM so that the size of the merged R-tree is much smaller (i.e., resulting in fewer disk I/Os), specifically in update-intensive workloads. By reducing the time for the merge operation, *UM+M* improves the overall performance of the update operations. Although both the *Eager* and *Validation* strategies also clean obsolete entries during merging, they have to validate objects using primary key/delete-key B-tree indexes on disk. The extra I/Os and search operations from these LSM B⁺-trees slow down these policies.

Refer to Figure 8b. The time for update processing is reduced significantly compared to the other clean-

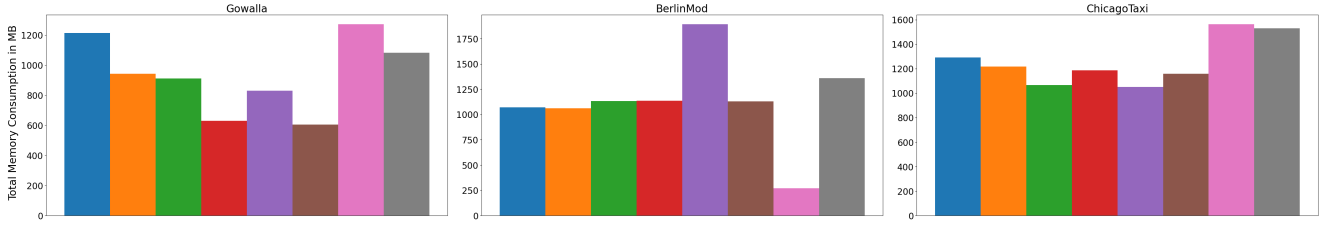


Fig. 9: Memory consumption for each dataset

ing strategies. While the Gowalla dataset consists of check-in locations of users and the ChicagoTaxi dataset collects pickup locations of taxi trips, and the BerlinMOD dataset tracks moving objects for each vehicle. As objects in BerlinMOD **continuously** move in the space covered by the R-tree, the travel distance between two consecutive data points of the same object is likely small, and the majority of the old and new location updates fall within the same R-tree nodes. As *UM+BV* executes an in-memory R-tree cleaning, the space used to be occupied by the obsolete objects get freed, and can be used for accommodating future inserts/updates. Thus, an unnecessary node split by obsolete objects is avoided. This results in the update performance improving by a factor of 2 for the BerlinMOD dataset. Refer to Figure 8 and Table 1. Observe that the effect of the Buffered and Vacuum Cleaning strategies (i.e., *UM+BV*) in all datasets as the frequency and the time for flushes reduce significantly compared to the other cleaning strategies. As *UM+BV* clears R-tree nodes for future updates to avoid node splits, the memory footprint of the in-memory R-tree is reduced and the in-memory component is less likely to reach the size threshold. Thus, the number of flushes is reduced and flushes have less overhead.

Figure 9 gives the total memory allocated to evaluate each strategy. Let n be the number of unique objects in each dataset, and m be the number of operations. Notice that the LSM-RUM tree does not incur much extra memory compared to the *Eager* and *Validation* strategies. This may seem counter-intuitive. However, while UM requires extra memory to buffer updates, it requires at most $O(n)$ extra memory because UM only needs 1 entry for each object and buffers all operations in the *ts* and *cnt* attributes. To the contrary, in the worst case scenario, the *Eager* strategy requires $O(m)$ space in total if every delete/update operation targets an object not in the in-memory R-tree but that is on disk. Similarly, in the worst case, the *Validation* strategy inserts an entry into the primary key index for each update ($O(m)$ space as well). Moreover, it takes memory to clean the obsolete entries, e.g., buffer pages, to load the delete-key B-tree and the primary key indexes

to validate the entries. *UM+BV* has different performance across datasets. Gowalla and ChicagoTaxi do not exhibit similar localities as in the location updates for BerlinMod. Thus, in this case, *UM+BV* does not clean as many entries as for BerlinMod. Therefore, the memory footprint for the in-memory R-tree is not reduced, and the Buffer Cleaning and Vacuum cleaning strategies have memory overheads (e.g., creating cursors to scan the R-tree nodes) that outweighs the gains. To the contrary, *UM+BV* cleans many obsolete entries in BerlinMod to ingest new updates, and reduces the number of in-memory R-tree splits and the R-tree’s memory footprint. Therefore, *UM+BV* significantly reduces the amount of memory used (it exhibits no flush, which means that the in-memory R-tree never reaches 250MB). *UM+F* reduces memory cost by filtering out obsolete entries on the fly while writing tuples to flush buffers. The *Eager* and *Validation* strategies flush by copying tuples into memory buffers without filtering. Thus, they use more memory in flush buffers in contrast to *UM+F*. *UM+M* does not have high memory overhead than the baseline except for BerlinMod. This is mostly due to the large amount of updates and invalidated entries in BerlinMod. Also, due to large amount of obsolete objects on disk, during a merge, more obsolete objects are found, and UM is updated accordingly. As obsolete objects being removed, UM removes its entries, and its size shrinks. This causes multiple in-memory hashmap rebalances, and thus allocating more memory. However, this higher memory usage is temporary as the memory used by the original buckets gets recycled later.

strategy	Gowalla		BerlinMOD		ChicagoTaxi	
	I/Os	CPU	I/Os	CPU	I/Os	CPU
Eager	312MB	84.08%	2682MB	78.08%	784MB	85.44%
Validation	401MB	56.8%	9677MB	71.04%	1706MB	67.44%
UM+FMBV	5126KB	84.56%	107KB	82.8%	1799KB	80%

Table 2: Profiling Result of LSM R-trees

We also profiled each LSM R-tree’s CPU and I/Os using JProfiler [33]. Due to its need of sudo privilege, we run it on a local machine with 8-core M1 CPU

and 16GM DRAM. The profiling results of the *Eager* strategy, the *Validation* strategy, and the LSM RUM-tree with all cleaning strategies are given in Table 2. To simplify the presentation, and to only include the I/Os incurred from the LSM R-tree, we only report the amount in bytes **written** by each LSM R-tree in the I/Os category. Also, at the end of the experiment, AsterixDB asks each LSM R-tree to flush the in-memory R-tree to disk. The I/O profiling includes this flush as well. The CPU load is scaled by 8 cores. From the table, observe the significant improvement in the amount of data written to disk during LSM flush and after each merge operation. They show the benefits of using a memory-based UM to buffer updates/deletes. Moreover, it shows the advantage of the cleaning policies. Buffered and Vacuum cleaning decrease the number of flushes to disk. The Clean Upon Flush reduces the size of the in-memory R-tree flushed to disk, further reducing the I/Os. The significant decrease in I/Os explains LSM RUM-tree’s high performance. The CPU profiling shows that the *Validation* strategy has the worst CPU utilization, which is expected due to its large I/Os. Despite its lower CPU usage in Gowalla and Chicago, it still outperforms the *Eager* strategy in these datasets.

6.3 Search Performance

After data ingestion is completed, we select 100 random query points from each dataset. For each query point, we create query regions (minimum x, and y, and maximum x and y as query rectangles) with different selectivities, and measure the average time to get the query results. As discussed by Luo and Carey [58], the *Validation* strategy is worse than the *Eager* strategy because of the extra validation steps. If needs to validate each query point in the query predicate range against the in-memory and disk primary key indexes. Figure 10 gives the search (query) performance on each dataset for different query selectivities. Overall, the UM strategy without any cleaning (i.e., *UM*) shows up to 62x better performance than the *Validation* strategy. While *UM* is comparable to the *Eager* strategy on Gowalla (in Figure 10a, it gets worse on the other datasets by one order of magnitude as in Figure 10b and Figure 10c). Although *UM* resides in memory, obsolete objects from a query require massive amount of time to validate their states. Instead, the *Eager* strategy always removes obsolete objects in the in-memory R-tree if exists, and performs Clean Upon Merge. Therefore, it is essential to have appropriate cleaning strategies as discussed in Section 4. Notice that *UM* shows better performance than the *Validation* strategy over all datasets because

its in-memory structure does not require I/O for validation.

Figure 10 illustrates the effect of the cleaning strategies on the query performance and the large performance benefits they bring. As discussed in Section 4, the cleaning strategies help reduce the size of UM, eliminate the obsolete objects, and hence reduce the fewer objects to be validated during runtime), and most importantly, shrink the size of the disk component R-trees. First, we focus on the effect of the Clean Upon Flush (i.e., *UM+F*). As discussed in Section 4.3, *UM+F* minimizes the number of obsolete objects in the R-tree by filtering them out before the flush operations. Thus, the size of the flushed R-tree into disk is smaller than the original in-memory R-tree. The more the obsolete objects that exist in the in-memory R-tree, the smaller the disk R-tree that gets flushed after being cleaned. As in Figure 10, *UM+F* outperforms the *Eager* and the *Validation* strategies, specifically by up to 5636x in the case of the BerlinMOD dataset and by up to 924x in the ChicagoTaxi dataset with large query ranges. Because the continuous movements of the objects in BerlinMOD makes most of the objects obsolete, the size of the R-trees in disk gets much smaller for *UM+F*, and requires fewer disk I/Os for search operations.

Next, we focus on the advantages of the Buffered and Vacuum Cleaning strategies (i.e., *UM+BV*) as in Figures 10a and 10b. Due to its ability to clean a node of the in-memory R-tree, the size of the R-tree does not increase significantly in update-intensive workloads, and hence results in minimized flush operations. Therefore, there is no flush/merge on *UM+BV* for the Gowalla and BerlinMOD datasets in Table 1 and a query operation will not incur disk I/Os. Thus, *UM+BV* shows orders of magnitude better performance than the *Eager* and *Validation* strategies for these datasets.

Because the update-intensive workloads make most disk objects obsolete, it is likely that the Clean Upon Merge (i.e., *UM+M*) cleans most of the obsolete objects in disk. When combined with the other strategies, the best performance is *UM+FMBV* as in Figure 10c, where among all the cleaning strategies, achieve over 1300x speedup over *UM* on larger selectivities. Putting everything together, *UM+FMBV* has up to 1876x and up to 249292x better query performance than *Eager* and *Validation* strategies, respectively, for large selectivities. Observe that both the in-memory and disk-side cleaning strategies are important for query performance; in-memory cleaning reduces the obsolete objects (i.e., smaller false-positive query results) and disk-side cleaning leads to fewer I/Os.

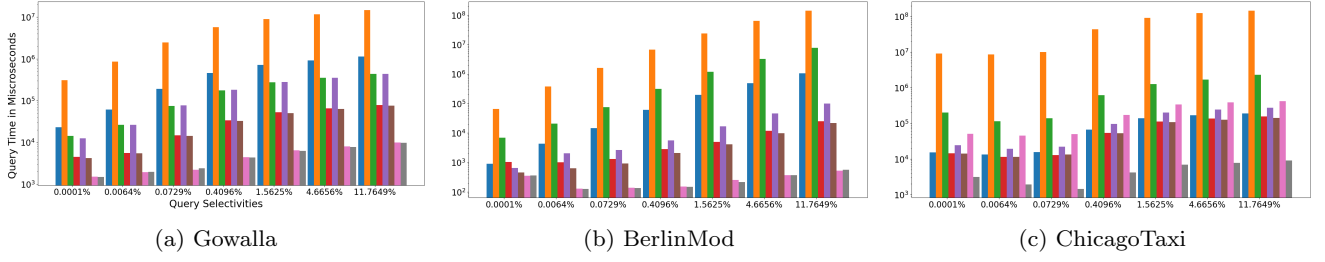


Fig. 10: Results of the query performance comparison study

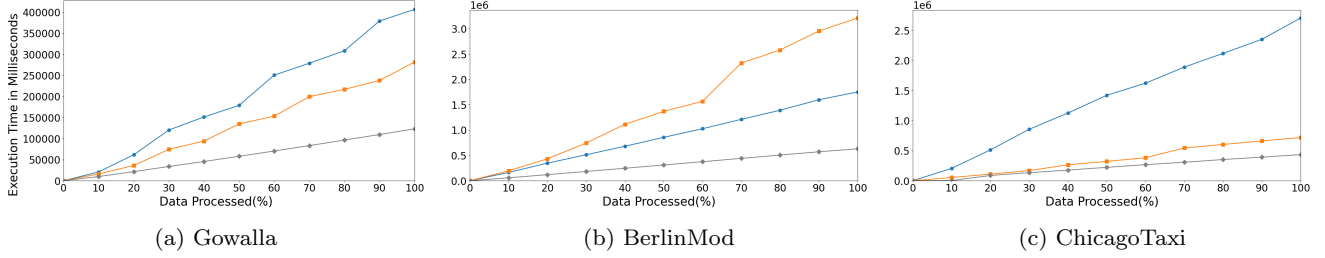


Fig. 11: Cumulative update processing time

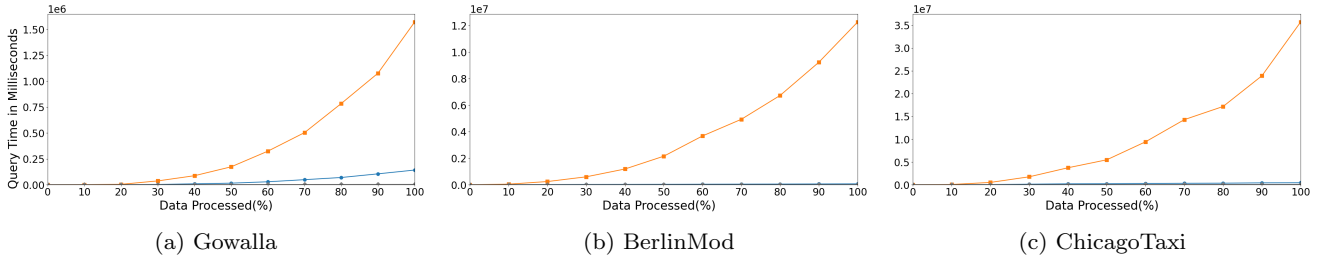


Fig. 12: Query processing time during data ingestion

6.4 Performance of Mixed Update and Query Workloads

In a real application, both update and query performance are important to provide stable services. For example, a shared vehicle service would *update* the current location of vehicles and *query* certain ranges to identify candidate vehicles for a passenger. Another example is a weather emergency application that would *update* users' locations and *query* the emergency area to send an emergency notification to the affected users. For the update/query mixed operations on LSM R-tree, query performance is degraded as the sizes of the in-memory R-tree grow and are flushed into disk.

In the previous experiment in Section 6.3, we execute random queries after all of the data ingestion has completed. In this experiment, we use the same three datasets, but interleave queries with the insert/update operations. To mimic a real update/query scenario using the dataset, we sort the data objects by their timestamps, as explained in Section 6.2, then choose 0.1% of

random objects in each dataset, and use their locations as query points rather than updating the locations of the objects. The range of the query varies uniformly up to 0.01% of the data space. In other words, Gowalla contains 6.4m objects in total so that there are about 6.4k (0.1%) queries interleaved with 6.3 Million (99.9%) updates. We choose this smaller number of queries because the *Validation* takes longer to finish processing more interleaved queries. The same applies to the BerlinMOD and ChicagoTaxi datasets. Figures 11, 12, and 13 give the cumulative update processing time, the cumulative query processing time for each data processing stage, and the cumulative processing time to execute both update and query operations for each dataset. We compare *UM+FMBV* with the *Eager* and the *Validation* strategies. *UM+FMBV* shows the best performance overall as in Sections 6.2 and 6.3.

From Figure 13, because of the high computational overhead of the validation step, the overall processing time of the *Validation* strategy is orders of magnitude longer than *UM+FMBV*. *UM+FMBV* also out-

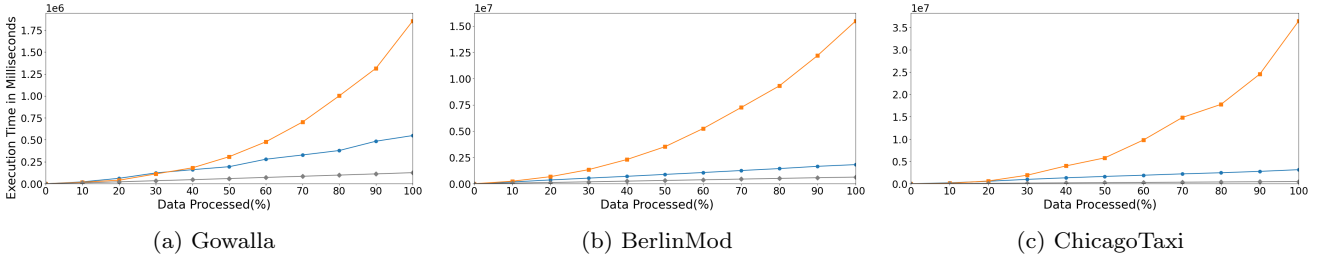


Fig. 13: Cumulative processing time, where 99.9% updates and 0.1% queries execute over each dataset

performs the *Eager* strategy by up 6.3x. The reasons for this difference in performance are as follows. First, the simple update procedures using UM result in improved performance as discussed in Section 3.2. The update operation in UM is straightforward, and does not incur extra maintenance cost. Second, the flush or merge operations slow down the update processing time. As in Figure 11, there are leaps on cumulative update processing time for the *Eager* and the *Validation* strategies due to the flush operations i.e., for the *Eager* strategy, flush executes during 20-30%, 50-60%, and 80-90% of the processed data, respectively). However, *UM+FMBV* does not incur the flush operation with the Gowalla dataset because of its ability to clean nodes only through the Buffered and Vacuum cleaning strategies. Also, observe that both cleaning strategies contribute to the improved update performance by making the in-memory R-tree avoid unnecessary node splits from obsolete objects. Even though *UM+FMBV* has flush operations with ChicagoTaxi (Figure 11’s ChicagoTaxi), the Clean Upon Flush makes the size of a flushed R-tree very small in update-intensive workloads, and thus the update processing time is stable. Lastly, the search performance under the *Eager* and the *Validation* strategies is degraded as updates are stacked in the LSM R-tree. The search processing time of the LSM R-tree increases due to the growing sizes of the R-tree(s), disk I/Os, and the calculation overhead of the validation steps.

In Figure 12, the search performance degrades in all strategies due to updates to the data. Although *UM+FMBV* does not incur disk I/O, the growing size of the R-tree in memory increases the search processing time and search may encounter false positive query points (obsolete objects). Also, disk I/Os are critical for search performance as in the *Eager* and the *Validation* strategies. The search processing time increases after flush operations take place because LSM R-tree has to incur I/Os for searches. Compared to the *Eager* and the *UM+FMBV* strategies, the *Validation* strategy shows worst search performance during updates, specifically in BerlinMod and ChicagoTaxi because of the valida-

tion steps using the primary key index. As the BerlinMOD and ChicagoTaxi datasets contain small numbers of distinct objects (2k and 5.2k unique objects, respectively), candidates that result from a search contain many obsolete objects, and validation using the *primary key index* results in higher computational overheads for the index traversal, and may lead to more disk I/O. Towards the end of the data processing, *UM+FMBV* achieves up to 56x and 10492x speedup on search processing time over the *Eager* and the *Validation* strategies. Within the UM, the memo structure is based on a hash map, and resides only in the memory layer so that the computational overhead of the validation steps is very small. Overall, the update and search performance of the LSM-RUM tree with cleaning strategies significantly outperform the conventional LSM R-tree strategies.

Real world workloads are not always uniform, and may contain hotspots. Therefore, we select around 20% of the queries, and create hotspot queries among them in the interleaved workloads. Their results are given in Figures 14, 15, and 16. These figures show similar trends and performance of the LSM RUM-Tree as in the case with uniform query selections. This demonstrates robustness of the LSM RUM-tree with respect to query distribution. Moreover, the LSM RUM-tree’s robustness can be seen throughout all figures in this experiment; Figures 11 — 16. The LSM RUM-tree’s execution time with respect to the data processing progress is almost always linear. It shows that the LSM RUM-tree does not experience similar performance drop in contrast to the other strategies. This demonstrates that the LSM-RUM tree is both performant and robust.

6.5 Multi-threads Support

As discussed in Section 5.1, we provide a mechanism to support concurrency control on the UM structure so that it can also run in a multi-threaded environment. To analyze its performance, we vary the number of threads running the *Eager* and *Validation* strategies, and compare the update performance with the LSM RUM-tree

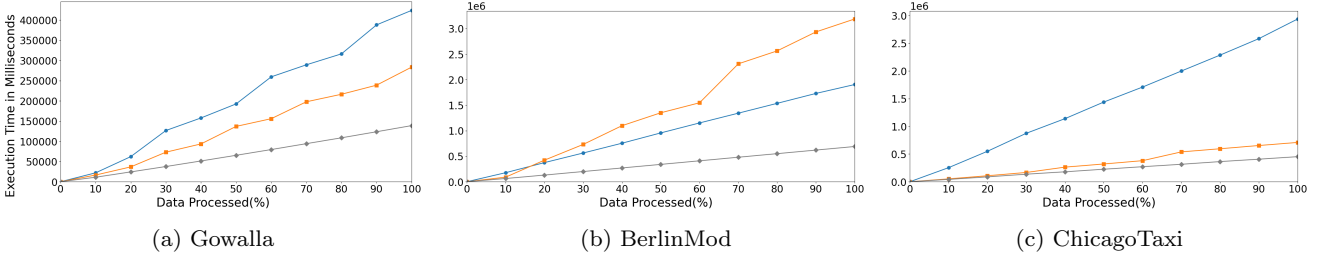


Fig. 14: Cumulative update processing time with hotspot queries

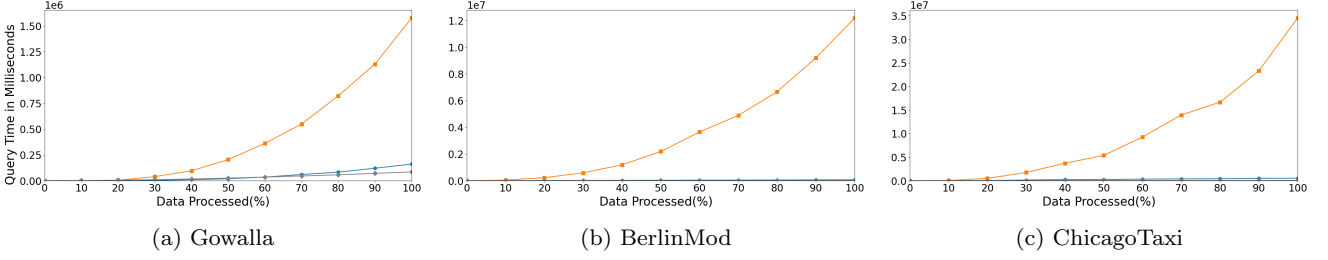


Fig. 15: Query processing time during data ingestion with hotspot queries

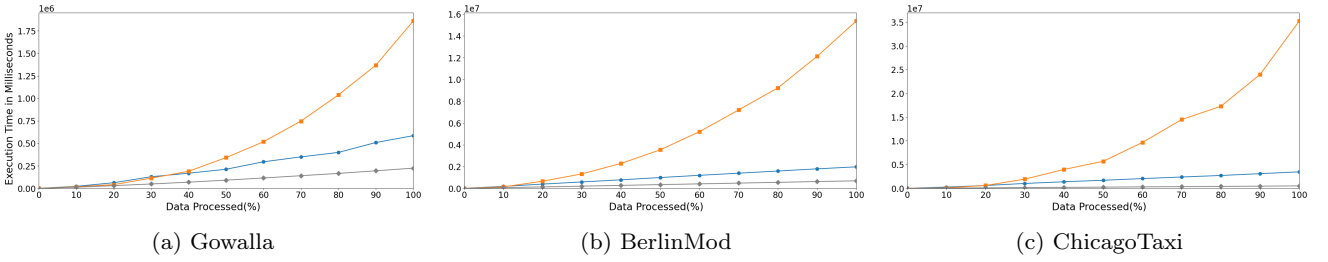


Fig. 16: Cumulative processing time, where 99.9% updates and 0.1% queries execute for each dataset with hotspot queries

using all the UM cleaning strategies. We use the same datasets, Gowalla, BerlinMOD, and ChicagoTaxi, and feed the data files for each thread running the update operations. We load all the data entries, and distribute them to all worker threads evenly before starting the execution. Therefore, we ensure there is no overhead in reading/fetching the operations from the input streams during execution.

From Figure 17, the LSM RUM-tree demonstrates superior performance and scalability over the baseline strategies. For the Gowalla dataset in Figure 17a, the LSM RUM-tree processes 158215 update operations/s with 8 concurrent working threads, which is 6.7x and 4x higher throughput than the *Eager* strategy (23643 update operations/s) and the *Validation* strategy (39232 update operations/s), respectively. Also, Figure 17 illustrates that the LSM RUM-tree has better scala-

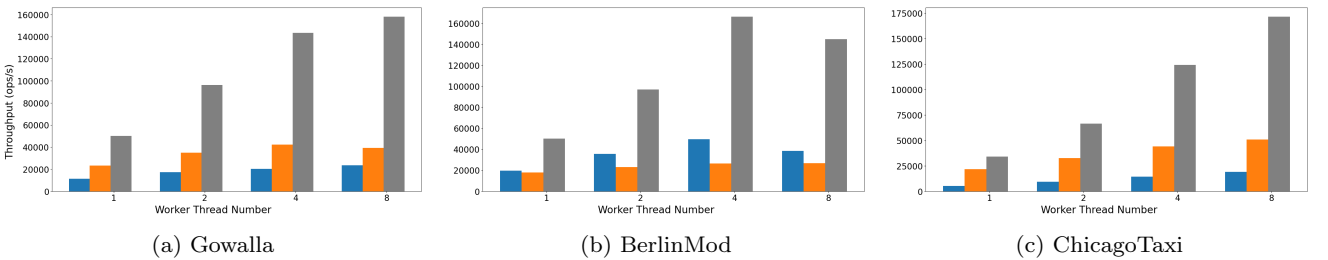


Fig. 17: Comparisons of update performance on multi-threads environment

bilities than *Eager* and *Validation* strategies. The results for the BerlinMod dataset (Figure 17b) and the ChicagoTaxi dataset (Figure 17c) are similar. With 8 threads, the LSM RUM-tree achieves a throughput of 145004 and 171726 update operations/s for the two datasets, respectively. In contrast, the *Validation* strategy achieves only 26689 and 50988 update operations/s and the *Eager* strategy achieves 38468 and 19168 update operations/s. The LSM RUM-tree clearly outperforms both the *Validation* and the *Eager* strategies using an LSM R-tree in both throughput and scalability.

7 Conclusions

Handling frequent updates in secondary LSM-based indexes is a crucial operation for high performance in update-heavy workloads. Careful design of update-tolerant LSM-based techniques is called for. This enables LSM-indexes to be highly performant not only for write-heavy workloads but also for update-heavy ones. In this paper, we introduce a new index structure, termed the LSM RUM-tree, that leverages an in-memory Update Memo structure (UM) with an LSM-based R-tree to enhance the update and search performance of update-intensive spatial data workloads. We illustrate how to utilize UM in the LSM RUM-tree for delete, update, and search operations. The in-memory UM structure provides efficient validation on query processing as well as simplified update operations. Making UM light-weight is important to be held in memory. To achieve this, we provide four UM cleaning strategies: Buffered Cleaning, Vacuum Cleaning, Clean Upon Flush, and Clean Upon Merge. These strategies not only clean UM entries to shrink its size, but also improve search performance as they also help shrink the size of the disk R-trees, and hence reduce the I/O overheads. The performance study demonstrates that the LSM RUM-tree handles update-intensive workloads efficiently, and outperforms the *Eager* and *Validation* strategies in the LSM R-tree. Also, the search performance is improved. We plan to expand this research to investigate how the update memo can enhance the performance of other implementations of LSM R-tree that contain different/orthogonal algorithms and optimizations. Also, we plan to adopt promising state-of-the-art LSM-tree techniques in our future designs of efficient LSM RUM-trees.

Acknowledgements Walid G. Aref acknowledges the support of the National Science Foundation under Grant Number IIS-1910216. Jianguo Wang acknowledges the support of the National Science Foundation under Grant Number IIS-2337806.

References

1. AsterixDB. URL <http://asterix.ics.uci.edu/>
2. Cassandra. URL <http://cassandra.apache.org/>
3. HBase. URL <http://hbase.apache.org/>
4. LevelDB. URL <https://github.com/google/leveldb/>
5. Oracle java 8 concurrenthashmap. URL <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ConcurrentHashMap.html>
6. RocksDB. URL <https://rocksdb.org/>
7. Scylladb. URL <https://www.scylladb.com/>
8. Influxdata. in-memory indexing and the time-structured merge tree (tsm). (2024). URL https://docs.influxdata.com/influxdb/v1/concepts/storage_engine/
9. Alkwaileet, W.Y., Alsubaiee, S., Carey, M.J.: An lsm-based tuple compaction framework for apache asterixdb. *Proc. VLDB Endow.* **13**(9), 1388–1400 (2020). DOI 10.14778/3397230.3397236. URL <https://doi.org/10.14778/3397230.3397236>
10. Alsubaiee, S., Altowim, Y., Altwaijry, H., Behm, A., Borkar, V., Bu, Y., Carey, M., Cetindil, I., Chee-langi, M., Faraaz, K., Gabrielova, E., Grover, R., Heilbron, Z., Kim, Y.S., Li, C., Li, G., Ok, J.M., Onose, N., Pirzadeh, P., Tsotras, V., Vernica, R., Wen, J., Westmann, T.: Asterixdb: a scalable, open source bdms. *Proc. VLDB Endow.* **7**(14), 1905–1916 (2014). DOI 10.14778/2733085.2733096. URL <https://doi.org/10.14778/2733085.2733096>
11. Alsubaiee, S., Behm, A., Borkar, V., Heilbron, Z., Kim, Y.S., Carey, M.J., Dreseler, M., Li, C.: Storage management in asterixdb. *Proceedings of the VLDB Endowment (PVLDB)* **7**(10), 841–852 (2014)
12. Alsubaiee, S., Carey, M.J., Li, C.: Lsm-based storage and indexing: An old idea with timely benefits. In: *Second International ACM Workshop on Managing and Mining Enriched Geo-spatial Data (GeoRich)*, pp. 1–6 (2015)
13. Apache: Accumulo. URL <https://accumulo.apache.org/>
14. Athanassoulis, M., Idreos, S.: Design tradeoffs of data access methods. In: *Proceedings of the 2016 International Conference on Management of Data, SIGMOD '16*, p. 2195–2200. Association for Computing Machinery, New York, NY, USA (2016). DOI 10.1145/2882903.2912569. URL <https://doi.org/10.1145/2882903.2912569>
15. Athanassoulis, M., Kester, M.S., Maas, L.M., Stoica, R., Idreos, S., Ailamaki, A., Callaghan, M.D.: Designing access methods: The rum conjecture. In: *International Conference on Extending Database Technology* (2016). URL <https://api.semanticscholar.org/CorpusID:15354110>
16. Beckmann, N., Kriegel, H.P., Schneider, R., Seeger, B.: The r*-tree: an efficient and robust access method for points and rectangles. In: *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data, SIGMOD '90*, p. 322–331. Association for Computing Machinery, New York, NY, USA (1990). DOI 10.1145/93597.98741. URL <https://doi.org/10.1145/93597.98741>
17. Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM* **13**(7), 422–426 (1970)
18. Cao, Z., Dong, S., Vemuri, S., Du, D.H.C.: Characterizing, modeling, and benchmarking rocksdb key-value workloads at facebook. In: *Proceedings of the 18th USENIX Conference on File and Storage Technologies, FAST'20*, p. 209–224. USENIX Association, USA (2020)

19. Chandramouli, B., Prasaad, G., Kossmann, D., Levandoski, J., Hunter, J., Barnett, M.: Faster: A concurrent key-value store with in-place updates. In: Proceedings of the 2018 International Conference on Management of Data, SIGMOD '18, p. 275–290. Association for Computing Machinery, New York, NY, USA (2018). DOI 10.1145/3183713.3196898. URL <https://doi.org/10.1145/3183713.3196898>
20. Chang, F., Dean, J., Ghemawat, S., Hsieh, W.C., Wallach, D.A., Burrows, M., Chandra, T., Fikes, A., Gruber, R.E.: Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)* **26**(2), 1–26 (2008)
21. Chatterjee, S., Jagadeesan, M., Qin, W., Idreos, S.: Cosine: a cloud-cost optimized self-designing key-value storage engine. *Proc. VLDB Endow.* **15**(1), 112–126 (2021). DOI 10.14778/3485450.3485461. URL <https://doi.org/10.14778/3485450.3485461>
22. Chen, S., Jensen, C.S., Lin, D.: A benchmark for evaluating moving object indexes. *Proceedings of the VLDB Endowment (PVLDB)* **1**(2), 1574–1585 (2008)
23. Chen, Y., Lu, Y., Yang, F., Wang, Q., Wang, Y., Shu, J.: Flatstore: An efficient log-structured key-value storage engine for persistent memory. *ASPLOS '20*, p. 1077–1091. Association for Computing Machinery, New York, NY, USA (2020). DOI 10.1145/3373376.3378515. URL <https://doi.org/10.1145/3373376.3378515>
24. Cho, E., Myers, S.A., Leskovec, J.: Friendship and mobility: user movement in location-based social networks. In: Proceedings of ACM International Conference on Knowledge Discovery and Data Mining (SIGKDD), pp. 1082–1090. ACM (2011)
25. Dai, Y., Xu, Y., Ganesan, A., Alagappan, R., Kroth, B., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H.: From wisckey to bourbon: a learned index for log-structured merge trees. In: Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation, OSDI'20. USENIX Association, USA (2020)
26. Dayan, N., Athanassoulis, M., Idreos, S.: Optimal bloom filters and adaptive merging for lsm-trees. *ACM Trans. Database Syst.* **43**(4) (2018). DOI 10.1145/3276980. URL <https://doi.org/10.1145/3276980>
27. Dayan, N., Idreos, S.: Dostoevsky: Better space-time trade-offs for lsm-tree based key-value stores via adaptive removal of superfluous merging. In: Proceedings of the 2018 International Conference on Management of Data, SIGMOD '18, p. 505–520. Association for Computing Machinery, New York, NY, USA (2018). DOI 10.1145/3183713.3196927. URL <https://doi.org/10.1145/3183713.3196927>
28. Dayan, N., Idreos, S.: The log-structured merge-bush & the wacky continuum. In: Proceedings of the 2019 International Conference on Management of Data, SIGMOD '19, p. 449–466. Association for Computing Machinery, New York, NY, USA (2019). DOI 10.1145/3299869.3319903. URL <https://doi.org/10.1145/3299869.3319903>
29. Dayan, N., Twitto, M.: Chucky: A succinct cuckoo filter for lsm-tree. In: Proceedings of the 2021 International Conference on Management of Data, SIGMOD '21, p. 365–378. Association for Computing Machinery, New York, NY, USA (2021). DOI 10.1145/3448016.3457273. URL <https://doi.org/10.1145/3448016.3457273>
30. DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Voshall, P., Vogels, W.: Dynamo: amazon's highly available key-value store. *ACM SIGOPS operating systems review* **41**(6), 205–220 (2007)
31. Düntgen, C., Behr, T., Güting, R.H.: Berlinmod: a benchmark for moving object databases. *The VLDB Journal* **18**(6), 1335 (2009)
32. Facebook: Memtable (2021). URL <https://github.com/facebook/rocksdb/wiki/MemTable>
33. ej-technologies GmbH: Java profiler - jprofiler (2024). URL <https://www.ej-technologies.com/products/jprofiler/overview.html>
34. Golan-Gueta, G., Bortnikov, E., Hillel, E., Keidar, I.: Scaling concurrent log-structured data stores. In: Proceedings of the Tenth European Conference on Computer Systems, EuroSys '15. Association for Computing Machinery, New York, NY, USA (2015). DOI 10.1145/2741948.2741973. URL <https://doi.org/10.1145/2741948.2741973>
35. Guttman, A.: R-trees: A dynamic index structure for spatial searching. In: Proceedings of the ACM International Conference on Management of Data (SIGMOD), pp. 47–57 (1984)
36. He, J., Chen, H.: An lsm-tree index for spatial data. *Algorithms* **15**(4) (2022). DOI 10.3390/a15040113. URL <https://www.mdpi.com/1999-4893/15/4/113>
37. Herlihy, M.: Wait-free synchronization. *ACM Transactions on Programming Languages and Systems (TOPLAS)* **13**(1), 124–149 (1991)
38. Hilbert, D.: Über die stetige abbildung einer linie auf ein flächenstück. *Mathematische Annalen* **38**, 459–460 (1891)
39. Huang, G., Cheng, X., Wang, J., Wang, Y., He, D., Zhang, T., Li, F., Wang, S., Cao, W., Li, Q.: X-engine: An optimized storage engine for large-scale e-commerce transaction processing. In: Proceedings of the 2019 International Conference on Management of Data, SIGMOD '19, p. 651–665. Association for Computing Machinery, New York, NY, USA (2019). DOI 10.1145/3299869.3314041. URL <https://doi.org/10.1145/3299869.3314041>
40. Huang, H., Ghandeharizadeh, S.: Nova-lsm: A distributed, component-based lsm-tree key-value store. In: Proceedings of the 2021 International Conference on Management of Data, SIGMOD '21, p. 749–763. Association for Computing Machinery, New York, NY, USA (2021). DOI 10.1145/3448016.3457297. URL <https://doi.org/10.1145/3448016.3457297>
41. Huynh, A., Chaudhari, H.A., Terzi, E., Athanassoulis, M.: Endure: a robust tuning paradigm for lsm trees under workload uncertainty. *Proc. VLDB Endow.* **15**(8), 1605–1618 (2022). DOI 10.14778/3529337.3529345. URL <https://doi.org/10.14778/3529337.3529345>
42. Idreos, S., Callaghan, M.: Key-value storage engines. In: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, SIGMOD '20, p. 2667–2672. Association for Computing Machinery, New York, NY, USA (2020). DOI 10.1145/3318464.3383133. URL <https://doi.org/10.1145/3318464.3383133>
43. Idreos, S., Dayan, N., Qin, W., Akmanalp, M., Hilgard, S., Ross, A.S., Lennon, J., Jain, V., Gupta, H., Li, D., Zhu, Z.: Design continuums and the path toward self-designing key-value stores that know and learn. In: Conference on Innovative Data Systems Research (2019). URL <https://api.semanticscholar.org/CorpusID:58013807>
44. Idreos, S., Zoumpatianos, K., Athanassoulis, M., Dayan, N., Hentschel, B., Kester, M.S., Guo, D., Maas, L.M., Qin, W., Wasay, A., Sun, Y.: The periodic table of data

- structures. *IEEE Data Eng. Bull.* **41**, 64–75 (2018). URL <https://api.semanticscholar.org/CorpusID:52190923>
45. Idreos, S., Zoumpatianos, K., Hentschel, B., Kester, M.S., Guo, D.: The data calculator: Data structure design and cost synthesis from first principles and learned cost models. In: *Proceedings of the 2018 International Conference on Management of Data, SIGMOD '18*, p. 535–550. Association for Computing Machinery, New York, NY, USA (2018). DOI 10.1145/3183713.3199671. URL <https://doi.org/10.1145/3183713.3199671>
 46. Kamel, I., Faloutsos, C.: Hilbert r-tree: An improved r-tree using fractals. In: *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, p. 500–509. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1994)
 47. Kannan, S., Bhat, N., Gavrilovska, A., Arpacı-Dusseau, A., Arpacı-Dusseau, R.: Redesigning lsms for nonvolatile memory with novelism. In: *Proceedings of the 2018 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '18*, p. 993–1005. USENIX Association, USA (2018)
 48. Kim, Y.S., Kim, T., Carey, M.J., Li, C.: A comparative study of log-structured merge-tree-based spatial indexes for big data. In: *33rd International Conference on Data Engineering (ICDE)*, pp. 147–150. IEEE (2017)
 49. Kondylakis, H., Dayan, N., Zoumpatianos, K., Palpanas, T.: Coconut palm: Static and streaming data series exploration now in your palm. *Proceedings of the 2019 International Conference on Management of Data* (2019). URL <https://api.semanticscholar.org/CorpusID:195259221>
 50. Kornacker, M., Mohan, C., Hellerstein, J.M.: Concurrency and recovery in generalized search trees. *ACM SIGMOD Record* **26**(2), 62–72 (1997)
 51. Kwon, D., Lee, S., Lee, S.: Indexing the current positions of moving objects using the lazy update r-tree. In: *3rd International Conference on Mobile Data Management (MDM)*, pp. 113–120. IEEE (2002)
 52. Lee, M.L., Hsu, W., Jensen, C.S., Cui, B., Teo, K.L.: Supporting frequent updates in r-trees: A bottom-up approach. In: *Proceedings of the International Conference on Very Large Data Bases (VLDB)*, pp. 608–619. Elsevier (2003)
 53. Lepers, B., Balmau, O., Gupta, K., Zwaenepoel, W.: Kvell: the design and implementation of a fast persistent key-value store. In: *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP '19*, p. 447–461. Association for Computing Machinery, New York, NY, USA (2019). DOI 10.1145/3341301.3359628. URL <https://doi.org/10.1145/3341301.3359628>
 54. Lepers, B., Balmau, O., Gupta, K., Zwaenepoel, W.: Kvell+: snapshot isolation without snapshots. In: *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation, OSDI'20*. USENIX Association, USA (2020)
 55. LinkedIn: Voldemort. URL <https://www.project-voldemort.com/voldemort/>
 56. Lu, L., Pillai, T.S., Gopalakrishnan, H., Arpacı-Dusseau, A.C., Arpacı-Dusseau, R.H.: Wisckey: Separating keys from values in ssd-conscious storage. *ACM Trans. Storage* **13**(1) (2017). DOI 10.1145/3033273. URL <https://doi.org/10.1145/3033273>
 57. Luo, C.: Breaking down memory walls in lsm-based storage systems. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, SIGMOD '20*, p. 2817–2819. Association for Computing Machinery, New York, NY, USA (2020). DOI 10.1145/3318464.3384399. URL <https://doi.org/10.1145/3318464.3384399>
 58. Luo, C., Carey, M.J.: Efficient data ingestion and query processing for lsm-based storage systems. *Proceedings of the VLDB Endowment (PVLDB)* **12**(5), 531–543 (2019)
 59. Luo, C., Carey, M.J.: On performance stability in lsm-based storage systems. *Proc. VLDB Endow.* **13**(4), 449–462 (2019). DOI 10.14778/3372716.3372719. URL <https://doi.org/10.14778/3372716.3372719>
 60. Mahmood, A., Aly, A., Kuznetsova, T., Basalamah, S., WG, A.: Disk-based indexing of recent trajectories. *ACM Transactions on Spatial Algorithms and Systems (TSAS)* **4**(3), 1–27 (2018)
 61. Mao, Q., Qader, M.A., Hristidis, V.: Comprehensive comparison of lsm architectures for spatial data. In: *2020 IEEE International Conference on Big Data (Big Data)*, pp. 455–460 (2020). DOI 10.1109/BigData50022.2020.9377919
 62. Mohan, C., et al.: ARIES/KVL: A key-value locking method for concurrency control of multiaction transactions operating on B-tree indexes. IBM Thomas J. Watson Research Division (1989)
 63. O'Neil, P., Cheng, E., Gawlick, D., O'Neil, E.: The log-structured merge-tree (lsm-tree). *Acta Informatica* **33**(4), 351–385 (1996)
 64. Pan, F., Yue, Y., Xiong, J.: dcompaction: Delayed compaction for the lsm-tree. *International Journal of Parallel Programming* **45**, 1310–1325 (2017). URL <https://api.semanticscholar.org/CorpusID:5609138>
 65. Peano, G.: Sur une courbe, qui remplit toute une aire plane. *Mathematische Annalen* **36**(1), 157–160 (1890)
 66. Portal, C.D.: Taxi Trips - 2019 (2019). URL <https://data.cityofchicago.org/Transportation/Taxi-Trips-2019/h4cq-z3dy>
 67. Qader, M.A., Cheng, S., Hristidis, V.: A comparative study of secondary indexing techniques in lsm-based nosql databases. In: *Proceedings of the 2018 International Conference on Management of Data, SIGMOD '18*, p. 551–566. Association for Computing Machinery, New York, NY, USA (2018). DOI 10.1145/3183713.3196900. URL <https://doi.org/10.1145/3183713.3196900>
 68. Raju, P., Kadekodi, R., Chidambaram, V., Abraham, I.: Pebblesdb: Building key-value stores using fragmented log-structured merge trees. In: *Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17*, p. 497–514. Association for Computing Machinery, New York, NY, USA (2017). DOI 10.1145/3132747.3132765. URL <https://doi.org/10.1145/3132747.3132765>
 69. Sarkar, S., Athanassoulis, M.: Dissecting, designing, and optimizing lsm-based data stores. pp. 2489–2497 (2022). DOI 10.1145/3514221.3522563
 70. Sarkar, S., Chen, K., Zhu, Z., Athanassoulis, M.: Compactionary: A dictionary for lsm compactions. In: *Proceedings of the 2022 International Conference on Management of Data, SIGMOD '22*, p. 2429–2432. Association for Computing Machinery, New York, NY, USA (2022). DOI 10.1145/3514221.3520169. URL <https://doi.org/10.1145/3514221.3520169>
 71. Sarkar, S., Papon, T.I., Staratzis, D., Athanassoulis, M.: Lethe: A tunable delete-aware lsm engine. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, SIGMOD '20*, p. 893–908. Association for Computing Machinery, New York, NY, USA (2020). DOI 10.1145/3318464.3389757. URL <https://doi.org/10.1145/3318464.3389757>
 72. Sarkar, S., Staratzis, D., Zhu, Z., Athanassoulis, M.: Constructing and analyzing the lsm compaction design

- p space.
- Proc. VLDB Endow.*
- 14**
- (11), 2216–2229 (2021). DOI 10.14778/3476249.3476274. URL
- <https://doi.org/10.14778/3476249.3476274>
73. Sears, R., Ramakrishnan, R.: blsm: a general purpose log structured merge tree. In: *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data, SIGMOD '12*, p. 217–228. Association for Computing Machinery, New York, NY, USA (2012). DOI 10.1145/2213836.2213862. URL <https://doi.org/10.1145/2213836.2213862>
 74. Shin, J., Wang, J., Aref, W.G.: The lsm rum-tree: A log structured merge r-tree for update-intensive spatial workloads. In: *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pp. 2285–2290. IEEE (2021)
 75. Silva, Y.N., Xiong, X., Aref, W.G.: The rum-tree: supporting frequent updates in r-trees using memos. *The VLDB Journal* **18**(3), 719–738 (2009)
 76. Song, M., Kitagawa, H.: Managing frequent updates in r-trees for update-intensive applications. *IEEE Transactions on Knowledge and Data Engineering* **21**(11), 1573–1589 (2009). DOI 10.1109/TKDE.2008.225
 77. Thonangi, R., Yang, J.: On log-structured merge for solid-state drives. In: *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*, pp. 683–694 (2017). DOI 10.1109/ICDE.2017.121
 78. Vinçon, T., Hardock, S., Riegger, C., Oppermann, J., Koch, A., Petrov, I.: Noftl-kv: Tackling write-amplification on kv-stores with native storage management (2018)
 79. Vo, A.V., Konda, N., Chauhan, N., Aljumaily, H., Laefer, D.F.: Lessons learned with laser scanning point cloud management in hadoop hbase. In: I.F.C. Smith, B. Dömer (eds.) *Advanced Computing Strategies for Engineering*, pp. 231–253. Springer International Publishing, Cham (2018)
 80. Wang, P., Sun, G., Jiang, S., Ouyang, J., Lin, S., Zhang, C., Cong, J.: An efficient design and implementation of lsm-tree based key-value store on open-channel ssd. In: *Proceedings of the Ninth European Conference on Computer Systems, EuroSys '14*. Association for Computing Machinery, New York, NY, USA (2014). DOI 10.1145/2592798.2592804. URL <https://doi.org/10.1145/2592798.2592804>
 81. Wu, X., Xu, Y., Shao, Z., Jiang, S.: Lsm-trie: an lsm-tree-based ultra-large key-value store for small data. In: *Proceedings of the 2015 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '15*, p. 71–82. USENIX Association, USA (2015)
 82. Xiong, X., Aref, W.G.: R-trees with update memos. In: *22nd International Conference on Data Engineering (ICDE)*, pp. 22–22 (2006)
 83. Xiong, X., Mokbel, M.F., Aref, W.G.: Lugrid: Update-tolerant grid-based indexing for moving objects. In: *7th International Conference on Mobile Data Management (MDM)*, pp. 13–13 (2006)
 84. Yao, T., Wan, J., Huang, P., He, X., Wu, F., Xie, C.: Building efficient key-value stores via a lightweight compaction tree. *ACM Trans. Storage* **13**(4) (2017). DOI 10.1145/3139922. URL <https://doi.org/10.1145/3139922>
 85. Zhang, W., Xu, Y., Li, Y., Li, D.: Improving write performance of lsm-tree-based key-value store. pp. 553–560 (2016). DOI 10.1109/ICPADS.2016.0079
 86. Zhang, W., Zhao, X., Jiang, S., Jiang, H.: Chameleondb: a key-value store for optane persistent memory. In: *Proceedings of the Sixteenth European Conference on Computer Systems, EuroSys '21*, p. 194–209. Association for Computing Machinery, New York, NY, USA (2021). DOI 10.1145/3447786.3456237. URL <https://doi.org/10.1145/3447786.3456237>
 87. Zhang, Z., Yue, Y., He, B., Xiong, J., Chen, M., Zhang, L., Sun, N.: Pipelined compaction for the lsm-tree. In: *2014 IEEE 28th International Parallel and Distributed Processing Symposium*, pp. 777–786 (2014). DOI 10.1109/IPDPS.2014.85
 88. Zhu, Y., Wang, S., Zhou, X., Zhang, Y.: Rum+-tree: A new multidimensional index supporting frequent updates. In: *The International Conference on Web-Age Information Management (WAIM)*, pp. 235–240 (2013)
 89. Zhu, Y., Wang, S., Zhou, X., Zhang, Y.: Rum+-tree: A new multidimensional index supporting frequent updates. pp. 235–240 (2013). DOI 10.1007/978-3-642-38562-9_24